Manual de despliegue, configuración y administración

Sistema de Gestión de Biblioteca

IZQUIERDO FP · CFGS Desarrollo de Aplicaciones Multiplataforma · Módulo 0488 Desarrollo de Interfaces · 2º DAM · Curso 2025-2026

Trabajo Nº1 · RA6 · Fase 4 (criterio f: manuales de instalación, configuración y administración)

Autores: Hugo Lopez y Daniel Carazo
Repositorio: https://github.com/vhugolm2023/Gestion-de-Biblioteca

1. Introducción y alcance

Este manual explica cómo poner en marcha la aplicación de gestión de biblioteca en un ordenador que no sea el nuestro. La aplicación está hecha en Java y funciona por consola: permite ver, buscar, añadir y borrar libros, y guarda los datos en un archivo de texto (`libros.txt`) o en una base de datos MySQL, según lo que se elija al arrancar.

Está pensado para alguien que sabe usar Windows y una terminal pero no conoce el proyecto. Nosotros lo hemos probado en una máquina virtual de VMware con Windows 10 recién instalado, siguiendo estos mismos pasos.

El manual cubre la instalación, la configuración y el mantenimiento (copias de seguridad, etc.). No explica cómo se usa la aplicación, eso va en el manual de usuario. Tampoco habla de la base de datos de la Fase 3 (`libreria`), porque no tiene nada que ver con esta aplicación.

Al terminar tendrás la aplicación funcionando con los dos repositorios, el de archivo y el de MySQL.

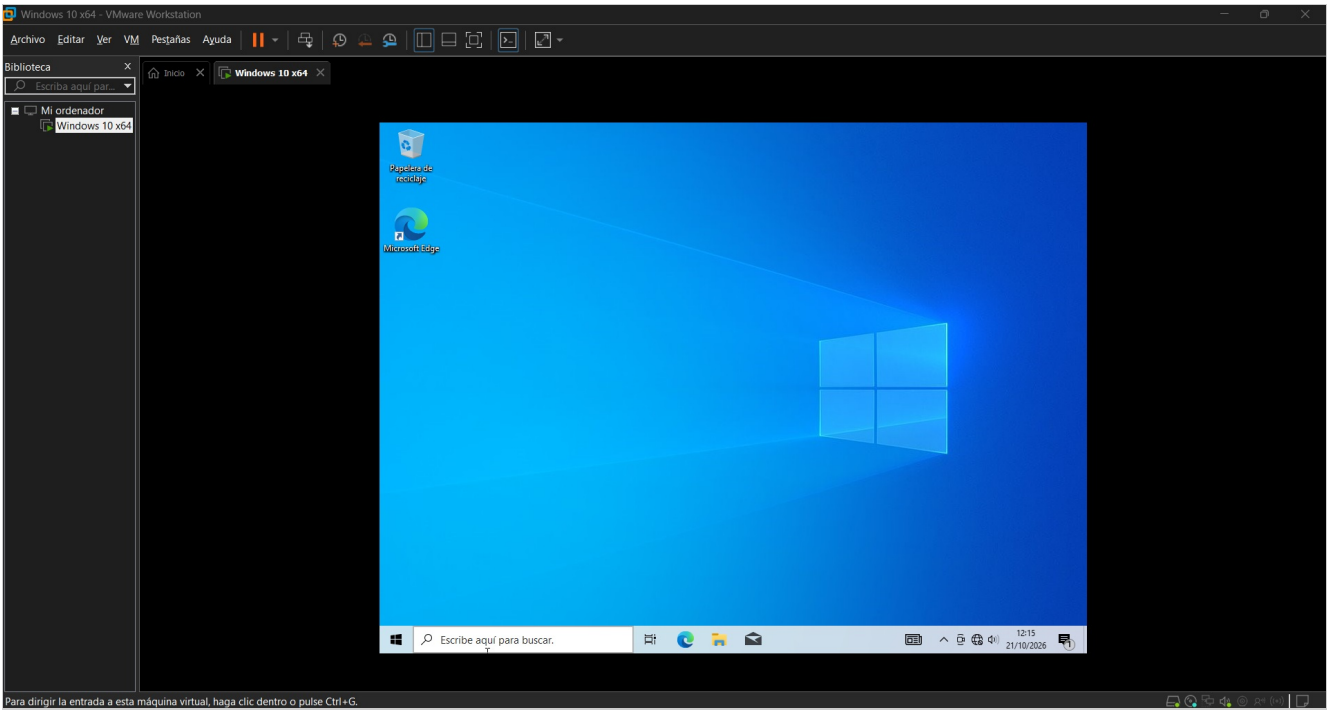
Todos los comandos se escriben en cmd (Símbolo del sistema), no en PowerShell. Más adelante se explica por qué.

2. Requisitos y entorno de pruebas

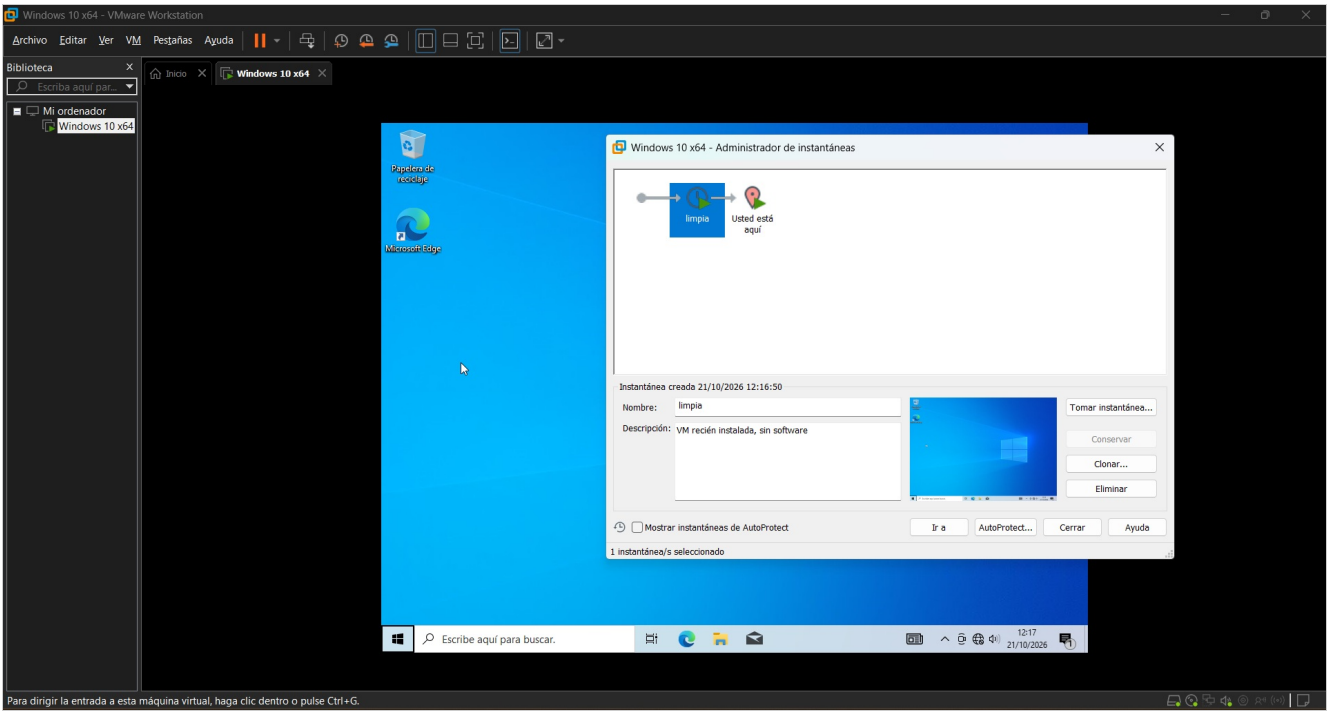
Software	Versión probada	Para qué
Windows	Windows 10 x64, versión 10.0.19045.3803 (VM en VMware Workstation)	Sistema operativo
JDK	Eclipse Temurin 26.0.2.1 (`openjdk version "26.0.2.1" 2026-08-18`)	Compilar y ejecutar
NetBeans	Apache NetBeans IDE 31	Abrir y compilar el proyecto (Ant)
MySQL Community Server	26.10.0 (`mysql  Ver 26.10.0 for Win64 on x86_64`); sirve desde la 8.0.16	Base de datos
Git	2.56.0.windows.2	Clonar el repositorio
mysql-connector-j	26.7.0 (se descarga aparte, apartado 6)	Driver para conectar con MySQL
dotenv-java	3.0.2 (se descarga aparte, apartado 6)	Leer el archivo `.env`

Hace falta Java 26 porque el proyecto está configurado con `javac.source=26`: el `.jar` que sale solo se ejecuta en Java 26 o más nuevo. De MySQL se pide la 8.0.16 o superior porque el script de la base de datos usa restricciones `CHECK`, que antes no se aplicaban.

La máquina virtual es un Windows 10 x64 en VMware Workstation, creado desde cero. Antes de instalar nada hicimos un snapshot llamado `limpia`, para poder volver al principio si algo salía mal.



Captura 01. Máquina virtual recién creada, con el escritorio vacío.



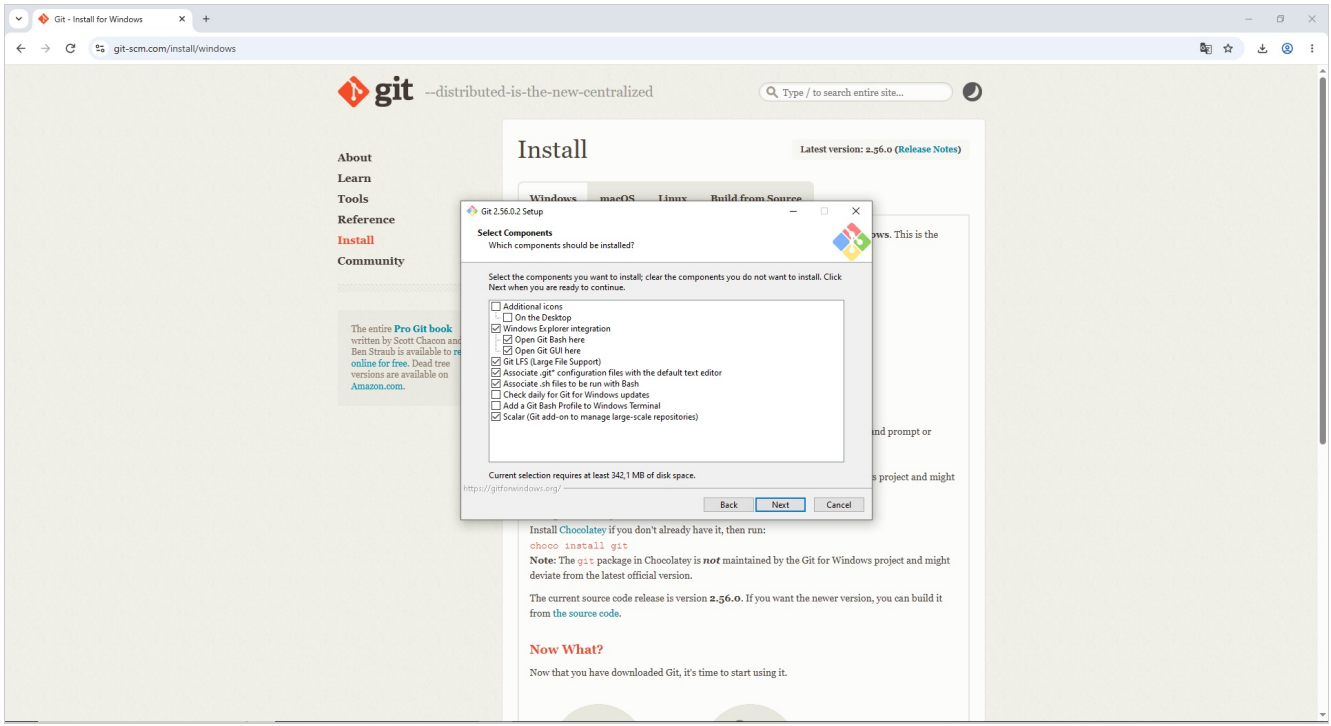
Captura 02. Snapshot «limpia» creado antes de instalar nada.

3. Instalación del software necesario

Se instalan por este orden. Después de cada instalación hay que abrir una terminal nueva para comprobar la versión, porque una que ya estuviera abierta no se entera de los cambios en el PATH.

3.1. Git

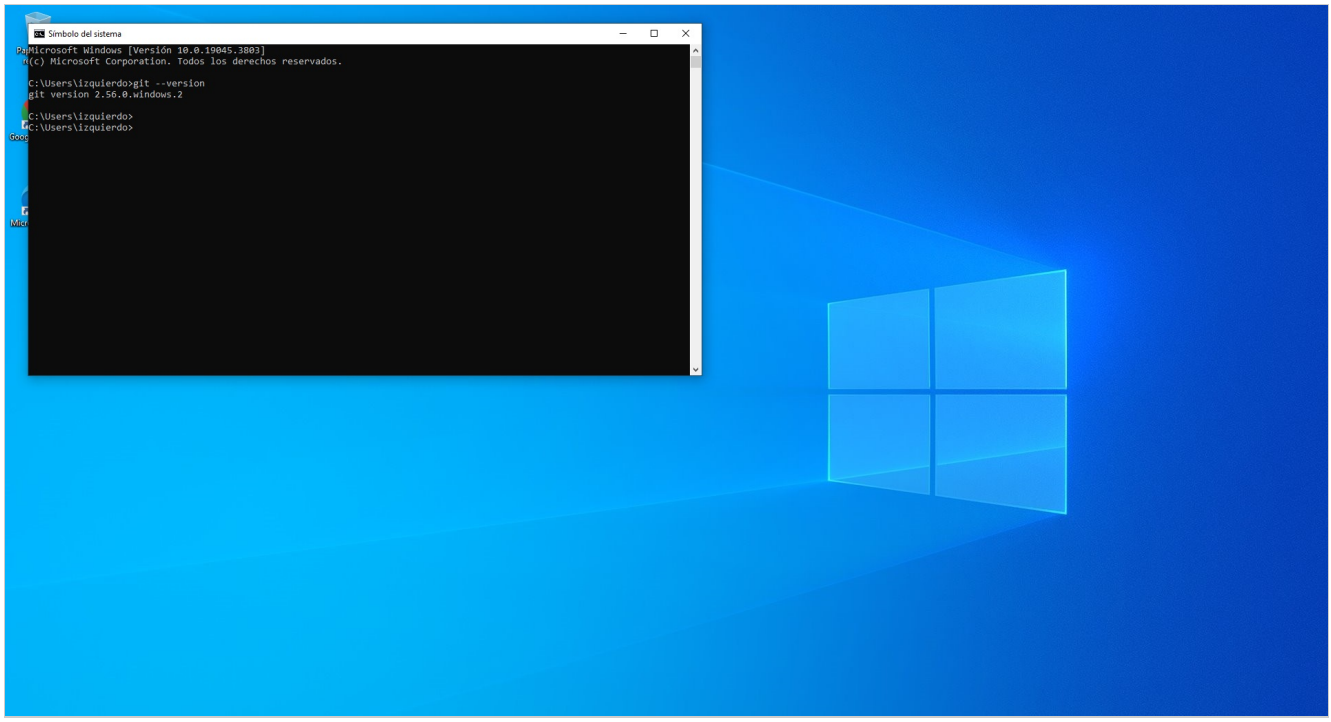
Descargamos el instalador de https://git-scm.com/ y lo dejamos todo por defecto (Next hasta el final).



Captura 03. Instalador de Git: pantalla de selección de opciones.

Para comprobarlo, en una cmd nueva:

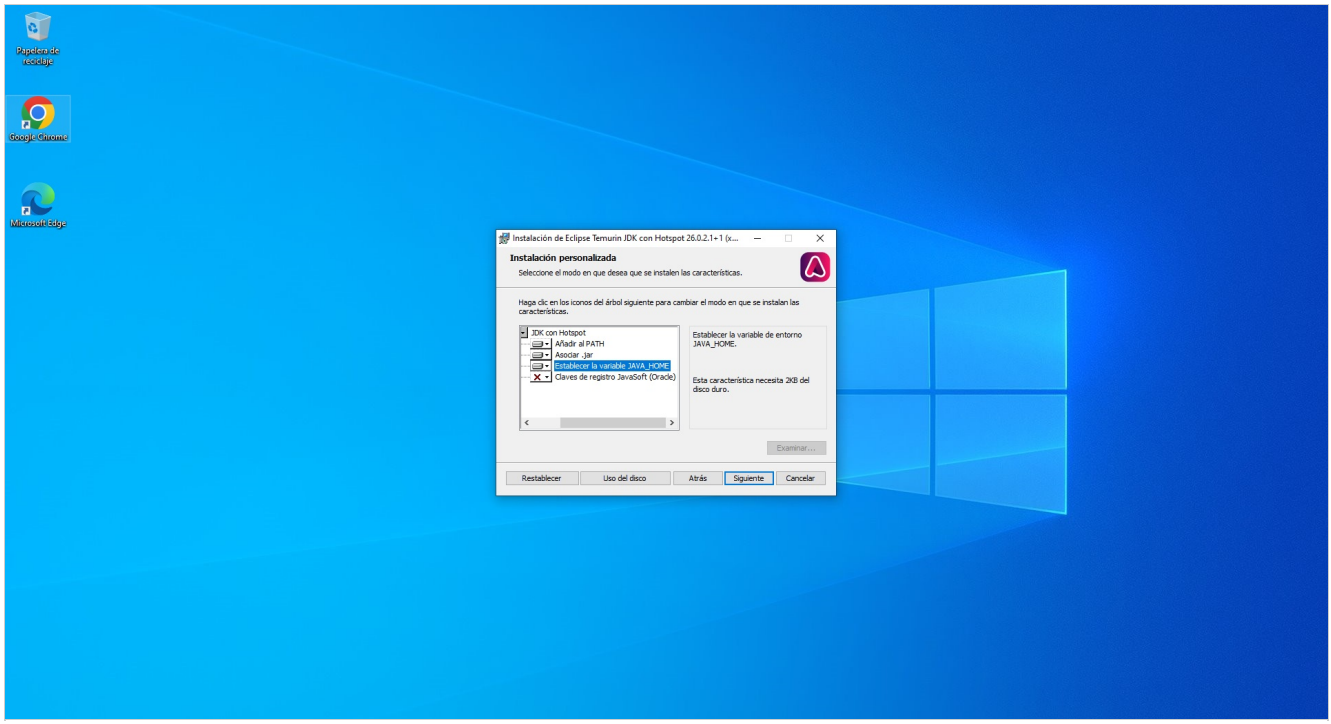
```
git --version
```



Captura 04. Salida de «git –version» en cmd.

3.2. JDK 26

Descargamos Eclipse Temurin 26 (el JDK, no el JRE) de https://adoptium.net/. En el instalador hay que activar la opción de añadir Java al PATH.

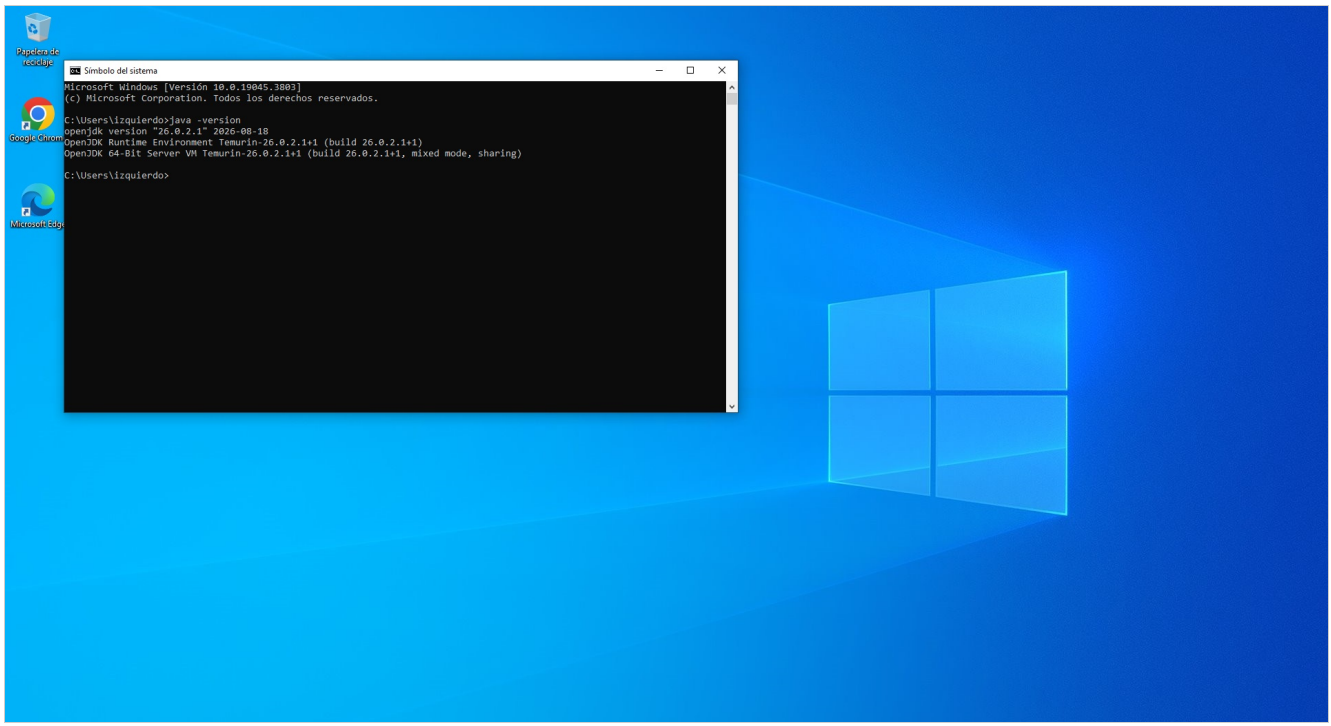


Captura 05. Instalador del JDK con la opción de añadir Java al PATH.

Para comprobarlo, en una cmd nueva:

```
java -version
```

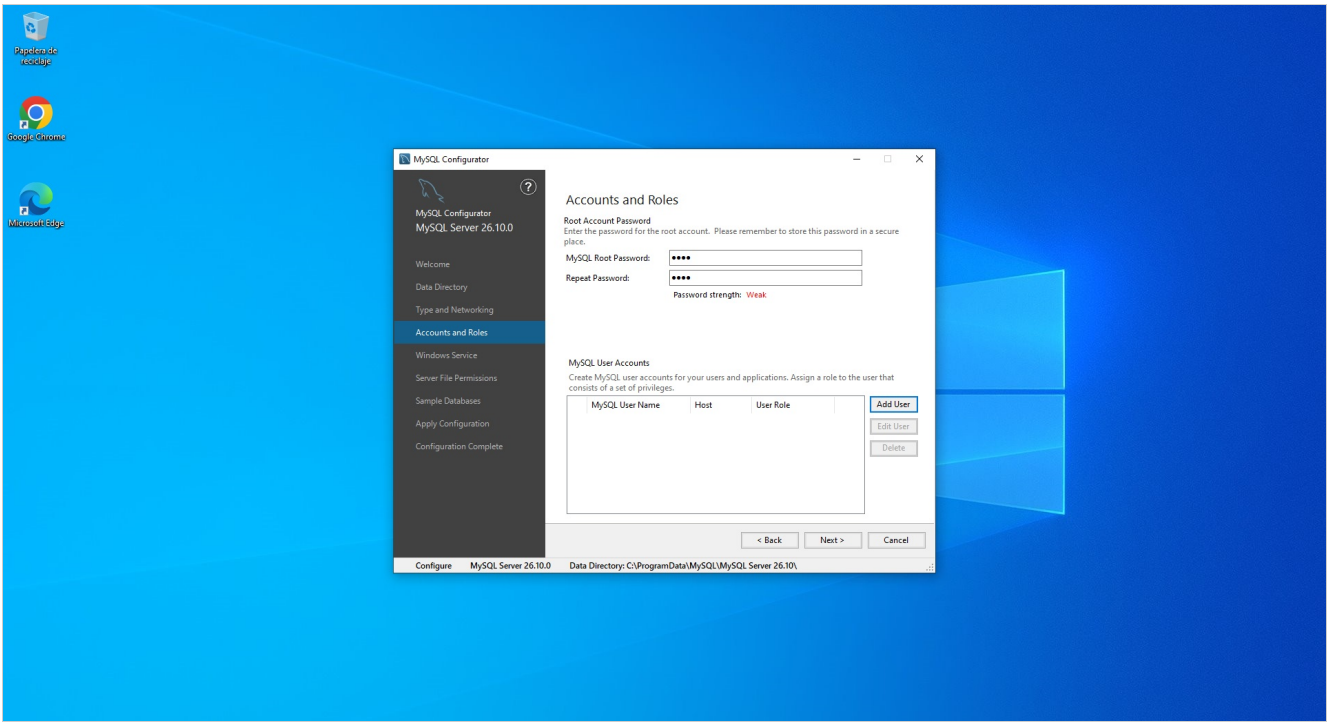
Tiene que salir la versión 26. Si sale otra, es que el PATH apunta a un Java más antiguo.



Captura 06. Salida de «java -version» en cmd.

3.3. MySQL Community Server

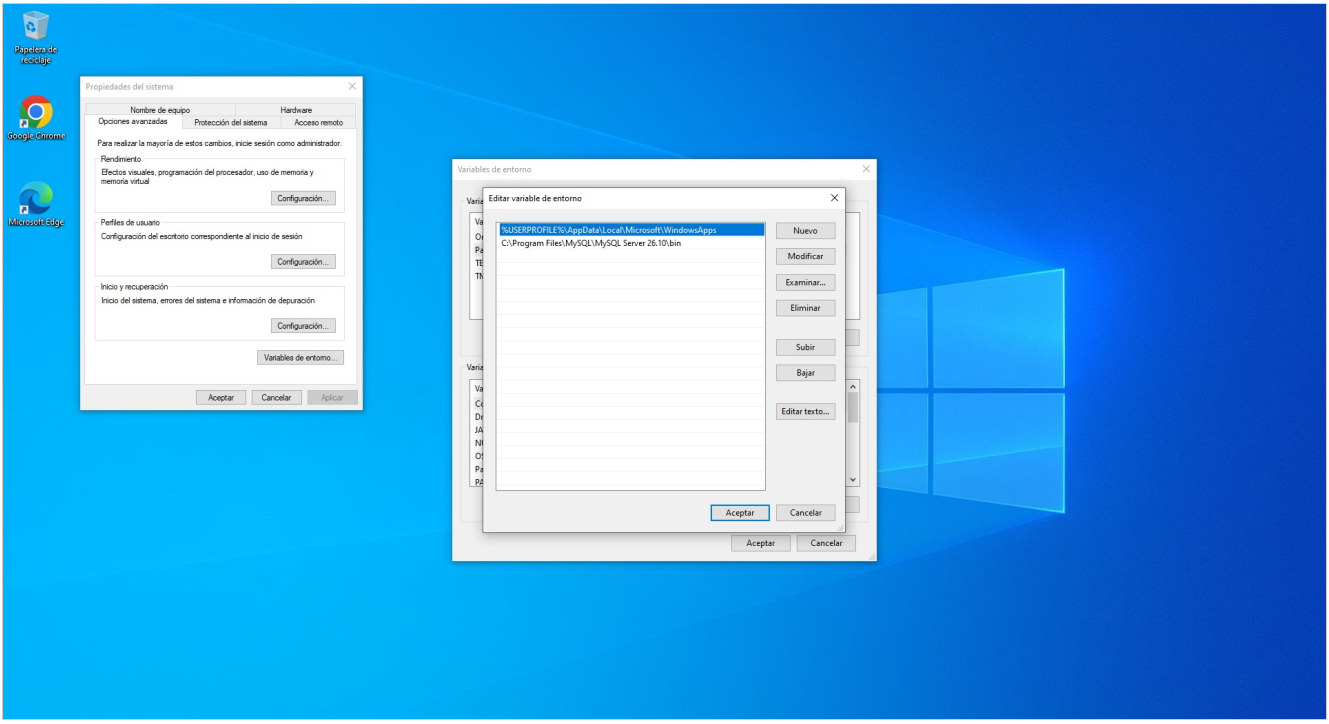
Lo descargamos de https://dev.mysql.com/downloads/mysql/ y elegimos instalar solo el servidor. En la pantalla Accounts and Roles hay que poner la contraseña del usuario `root`. Conviene apuntarla, porque después hace falta en el archivo `.env`.



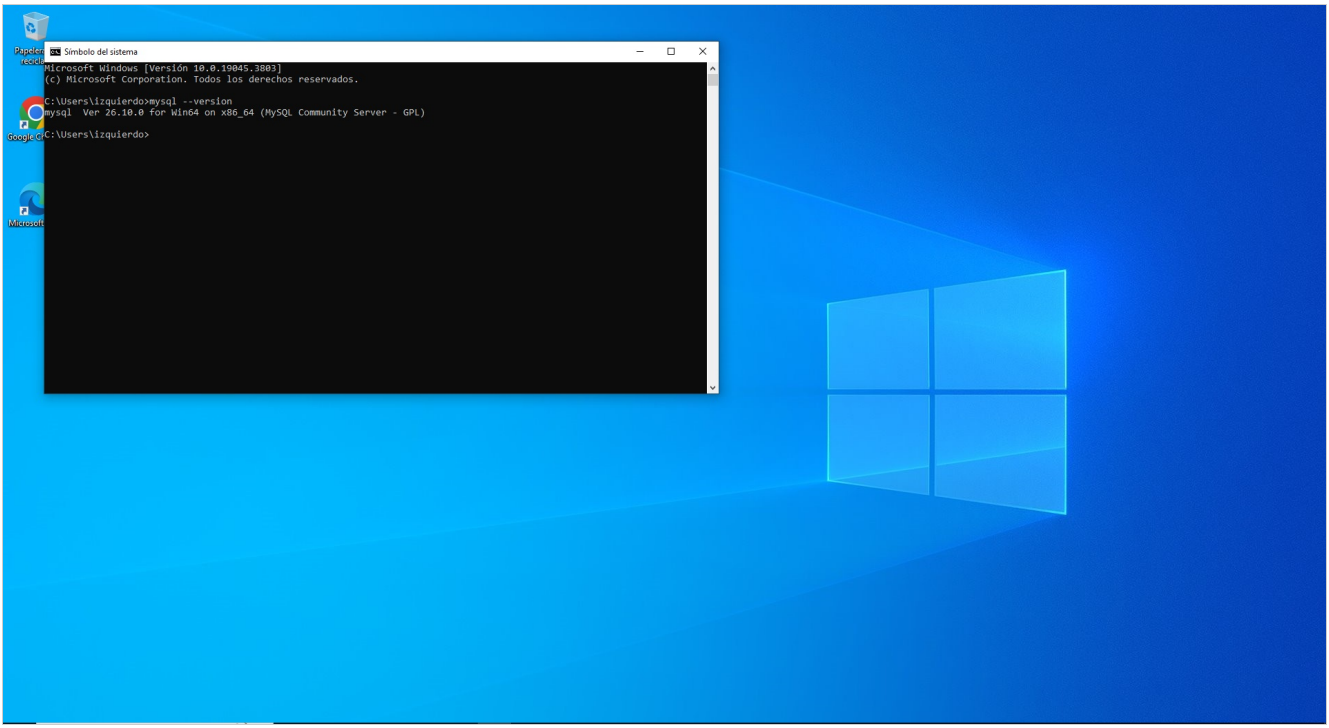
Captura 07. Instalador de MySQL: pantalla donde se define la contraseña de root (taparla en la captura).

Al comprobar la versión nos pasó que Windows no reconocía el comando (la captura 48 del apartado 12). El instalador de MySQL no añade su carpeta `bin` al PATH, así que hay que hacerlo a mano: Configuración avanzada del sistema → Variables de entorno → Path → Editar → Nuevo, y pegar la ruta, que en nuestro caso fue `C:\Program Files\MySQL\MySQL Server 26.10\bin`. Después se abre otra cmd y se prueba:

```
mysql --version
```

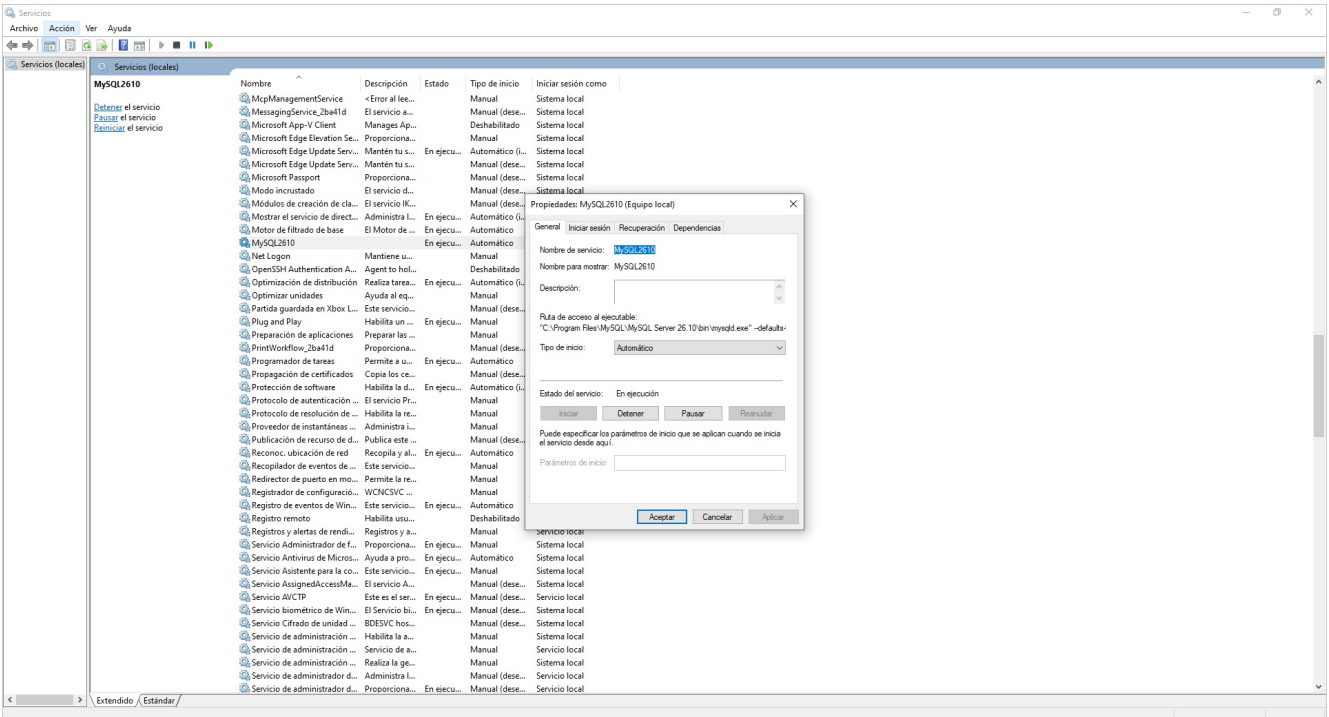


Captura 09. Variable de entorno Path con la carpeta bin de MySQL añadida.



Captura 08. Salida de «mysql –version» en cmd.

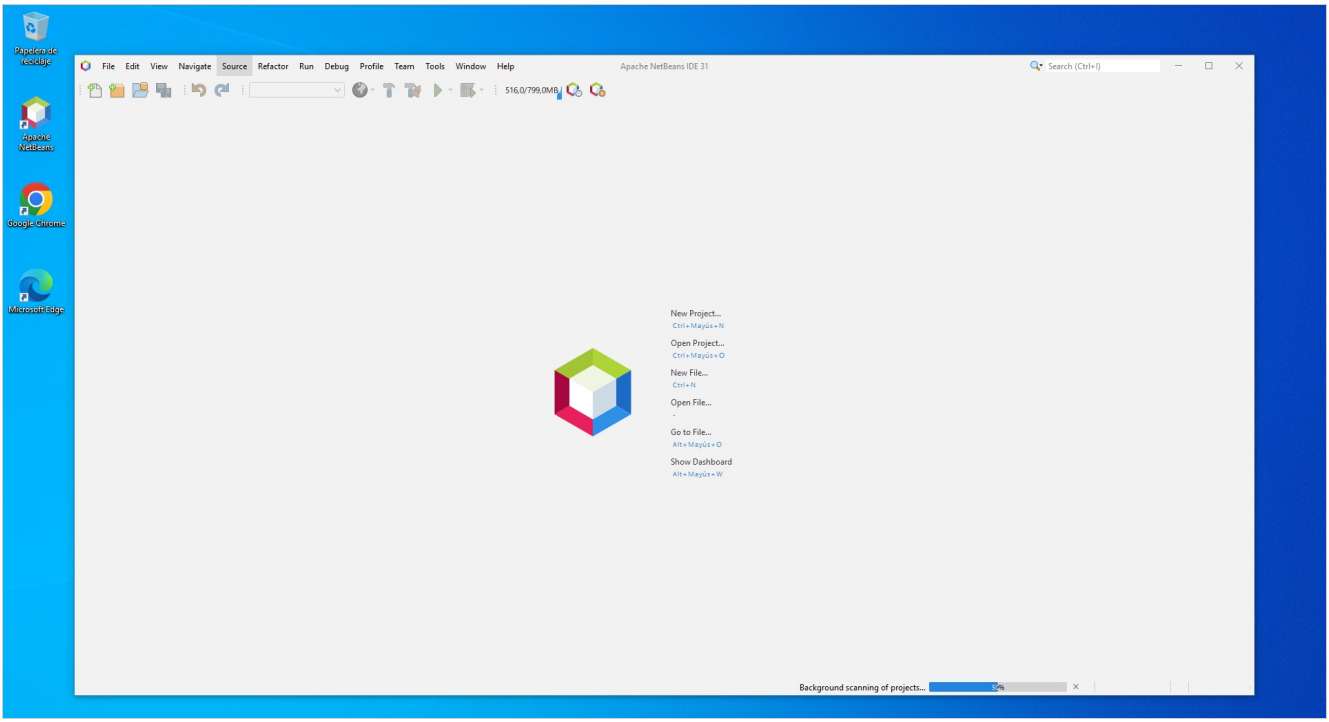
Para ver que el servidor está arrancado, se abre `services.msc` y se busca el servicio de MySQL. En nuestro caso se llama `MySQL2610` (el nombre cambia con la versión) y estaba en ejecución y en inicio automático.



Captura 10. Servicio de MySQL en ejecución en services.msc.

3.4. NetBeans 31

Lo descargamos de https://netbeans.apache.org/ y lo instalamos con las opciones por defecto. Al abrirlo, en Tools → Java Platforms comprobamos que aparecía el JDK 26.



Captura 11. NetBeans abierto tras la instalación.

4. Estructura del repositorio

```
Gestion-de-Biblioteca/
├── src/gestiondebiblioteca/   Código fuente Java
│   ├── GestionDeBiblioteca.java     Clase principal (menú)
│   ├── Libro.java                   Modelo
│   ├── LibroRepository.java         Interfaz de los repositorios
│   ├── LibroRepositoryArchivo.java  Repositorio de archivo de texto
│   ├── LibroRepositoryMySQL.java    Repositorio MySQL (JDBC)
│   └── ConexionBD.java              Conexión a MySQL (lee el .env)
├── sql/
│   └── init.sql               Crea la BD, la tabla y los datos de ejemplo
├── libros.txt                 Archivo inicial del repositorio de texto
├── docs/                      Documentación del despliegue
│   ├── manual_despliegue.pdf
│   └── imagenes/              Capturas de este manual
├── Entregafase3/              Documentación de la Fase 3 (otra base de datos)
├── nbproject/                 Configuración del proyecto NetBeans (Ant)
├── build.xml                  Script de compilación de Ant
├── manifest.mf                Manifiesto del .jar
├── .env                       Datos de conexión a MySQL (los de los autores)
├── .env.example               Plantilla con valores de ejemplo
├── .gitignore                 Archivos que no se suben a GitHub
└── README.md                  Resumen de instalación
```

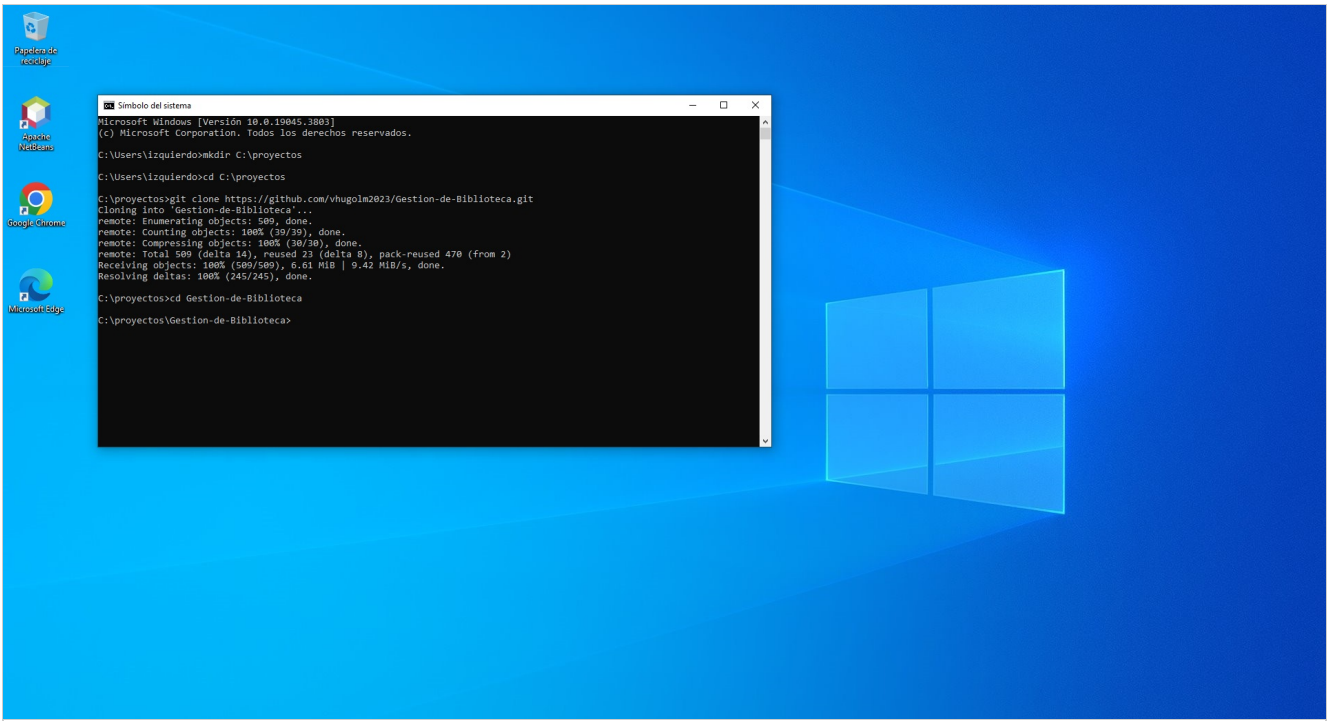
Las librerías (`.jar`) no están en el repositorio: se descargan aparte y se enlazan desde NetBeans (apartado 6).

El repositorio incluye un `.env` con los datos de conexión que usamos nosotros (usuario y contraseña de nuestro MySQL), así que hay que editarlo con los de cada equipo (apartado 8). El `.env.example` es la plantilla equivalente. Las carpetas `build/` y `dist/` no vienen al clonar, se crean al compilar.

5. Obtención del código

Se elige una carpeta de trabajo y se clona el repositorio con Git, en cmd:

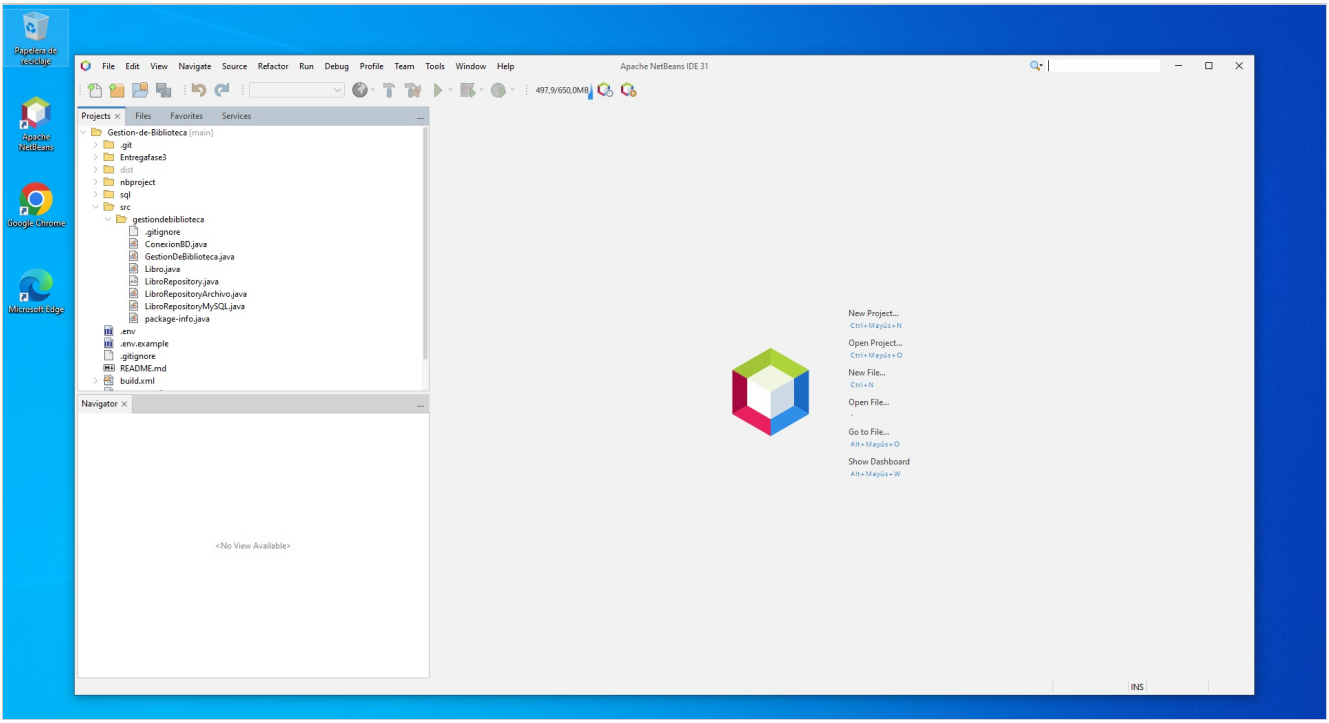
```
cd C:\proyectos
git clone https://github.com/vhugolm2023/Gestion-de-Biblioteca.git
```



Captura 12. «git clone» terminado correctamente.

Si no se quiere usar Git, también vale el botón Code → Download ZIP de GitHub, pero así se pierde el historial y no se puede actualizar con `git pull`.

Después se abre NetBeans, File → Open Project, y se elige la carpeta `Gestion-de-Biblioteca`. Es un proyecto Ant, no Maven. En Source Packages aparece el paquete `gestiondebiblioteca` con sus clases.



Captura 13. Proyecto abierto en NetBeans con el árbol de paquetes visible.

A nosotros NetBeans no nos mostró ningún aviso al abrirlo, pero el proyecto todavía no compila: faltan las librerías. Eso se arregla en el apartado siguiente.

6. Librerías del proyecto

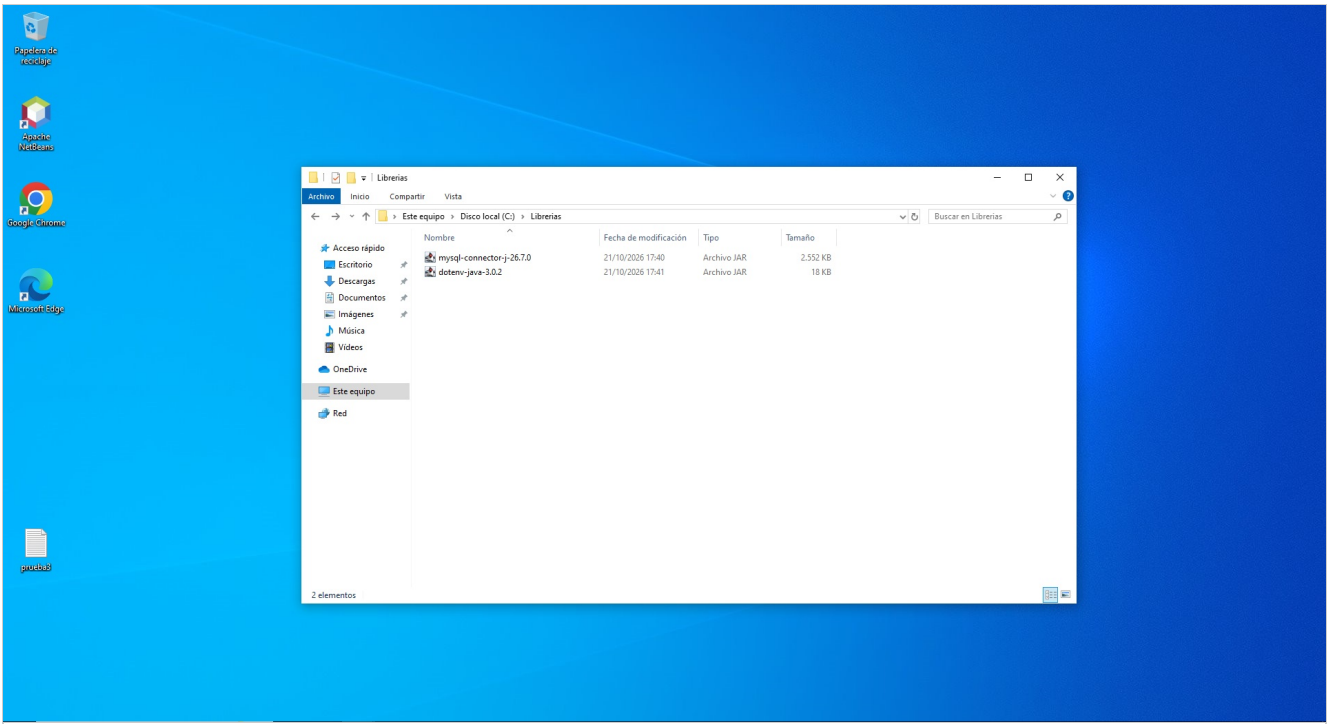
El proyecto usa Ant, no Maven, así que nadie descarga las librerías por nosotros. Los `.jar` no están en el repositorio y hay que bajarlos y enlazarlos desde NetBeans.

Librería	Versión	Para qué sirve	Dónde descargarla (Maven Central)
MySQL Connector/J	26.7.0	Driver para que Java hable con MySQL	https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/26.7.0/ (`mysql-connector-j-26.7.0.jar`)
dotenv-java	3.0.2	Lee las variables del archivo `.env`	https://repo1.maven.org/maven2/io/github/cdimascio/dotenv-java/3.0.2/ (`dotenv-java-3.0.2.jar`)

El proyecto original se hizo con la versión 3.2.0 de dotenv-java, pero en la máquina virtual lo probamos con la 3.0.2 y funciona igual.

6.1. Descargar los `.jar`

Creamos una carpeta fuera del proyecto, por ejemplo `C:\librerias`, y descargamos ahí los dos archivos `.jar` (solo el `.jar`, no los `-sources` ni los `-javadoc`).



Captura 14. Carpeta C:con los dos .jar descargados.

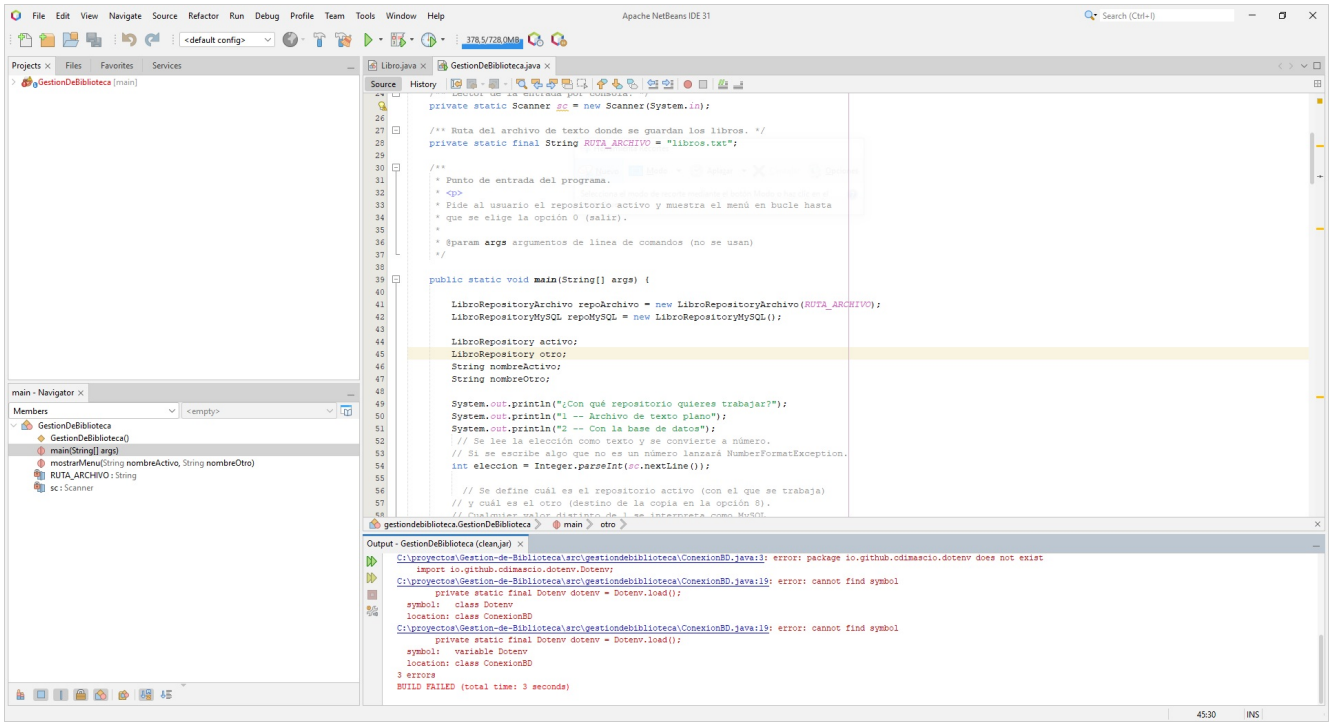
6.2. El proyecto no compila

El proyecto guarda en `nbproject/project.properties` las rutas de las librerías tal y como estaban en los ordenadores de quienes lo hicieron (por ejemplo `C:\Users\dan38\Downloads\librerias\...`). En otro equipo esas rutas no existen. Se puede ver con este comando, dentro de la carpeta del proyecto:

```
findstr /n "file.reference" nbproject\project.properties
```

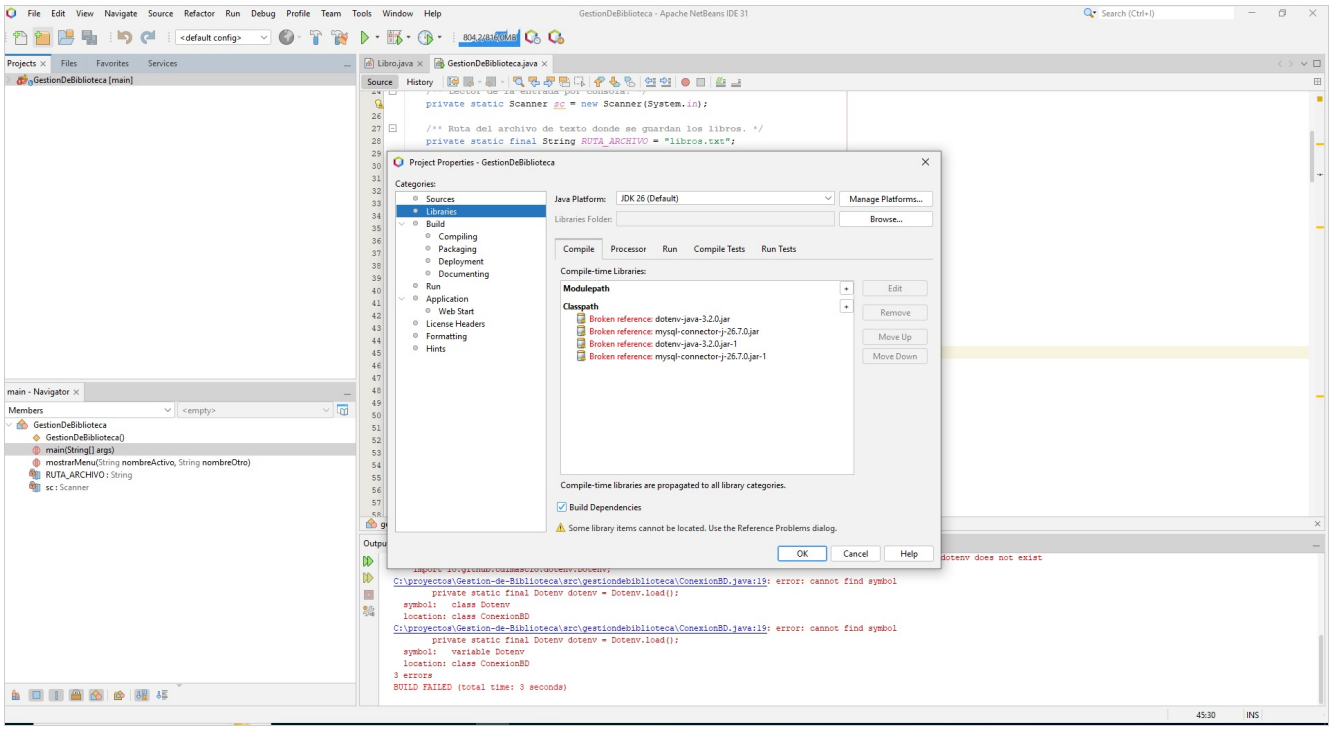
Si en NetBeans hacemos clic derecho sobre el proyecto → Clean and Build, falla con este error:

```
error: package io.github.cdimascio.dotenv does not exist
BUILD FAILED
```



Captura 15. Clean and Build sin enlazar las librerías: panel Output con «package io.github.cdimascio.dotenv does not exist» y BUILD FAILED.

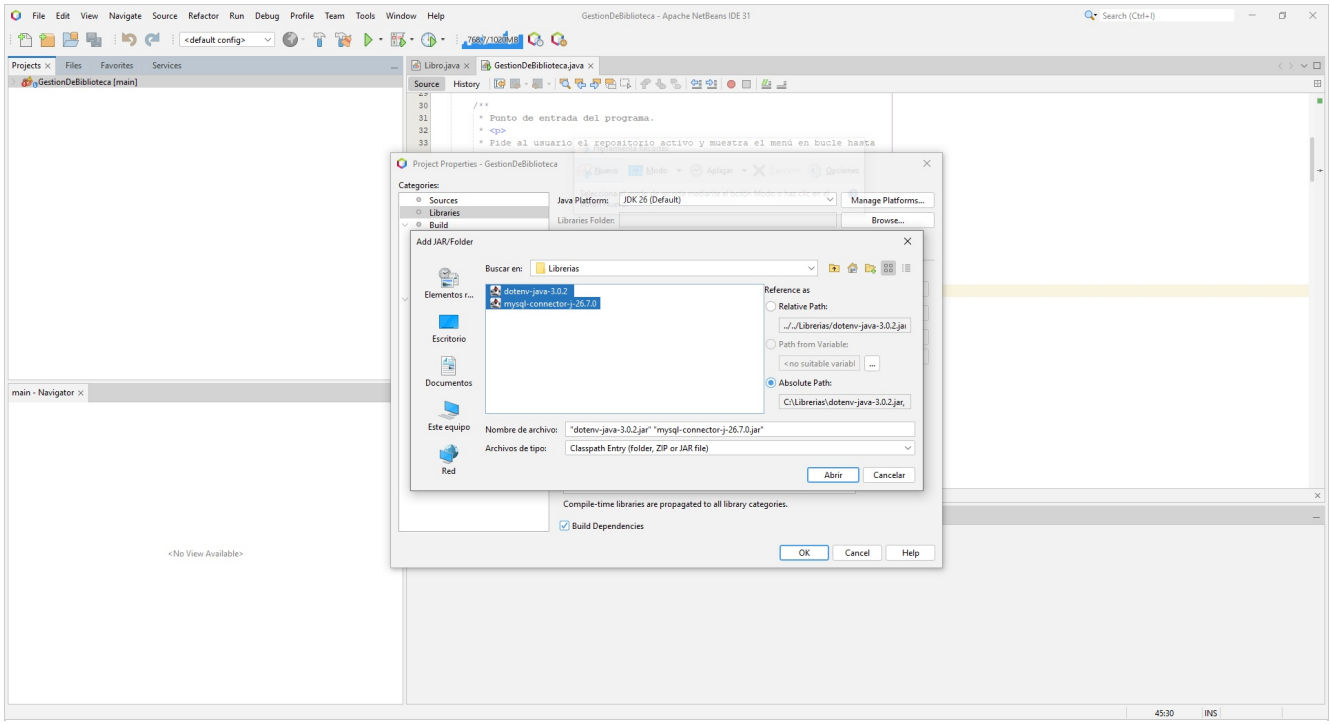
En Properties → Libraries se ven las entradas en rojo, porque apuntan a archivos que no existen:



Captura 15a. Properties → Libraries antes de enlazarlas (entradas apuntando a rutas que no existen).

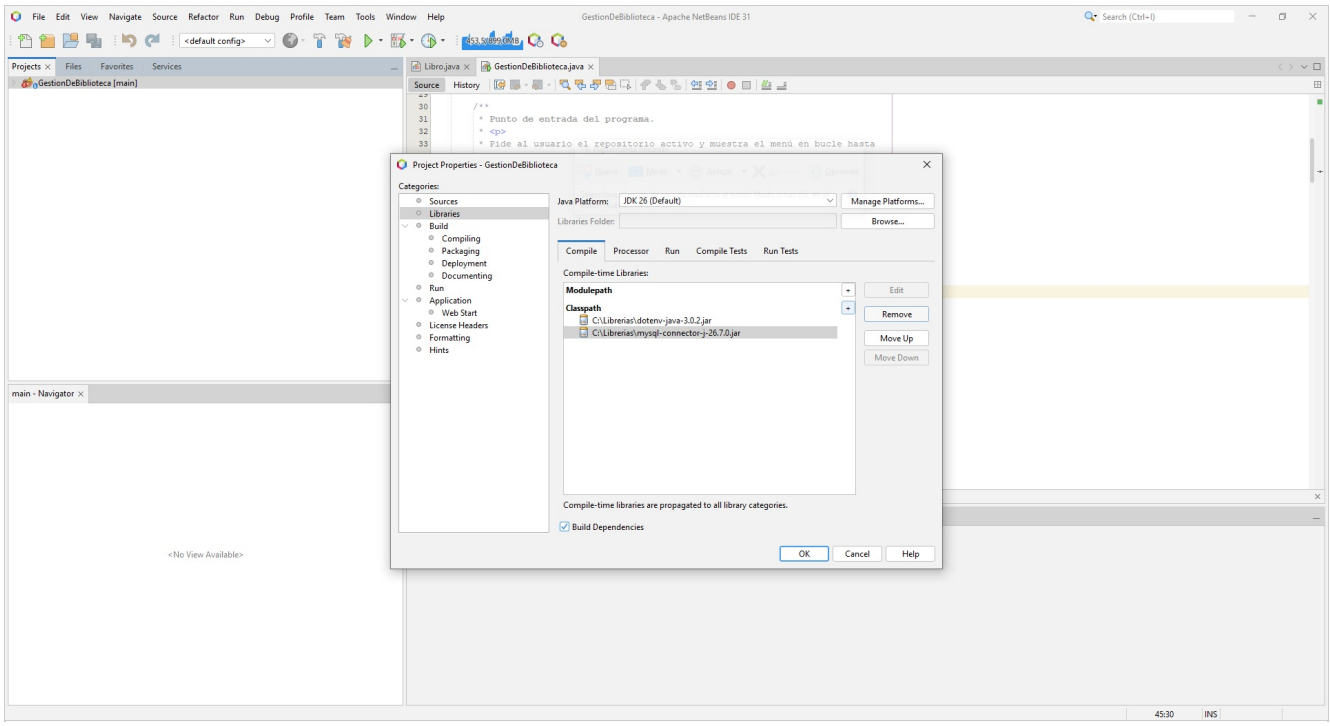
6.3. Enlazar las librerías

En Properties → Libraries → Compile hay cuatro entradas rotas (dos por cada librería). Las seleccionamos todas y pulsamos Remove. Después, con el botón «+» → Add JAR/Folder, elegimos los dos `.jar` de `C:\librerias`.



Captura 15b. Diálogo Add JAR/Folder con los .jar de C:seleccionados.

Al aceptar, las dos librerías aparecen ya sin errores:



Captura 15c. Properties → Libraries con las dos librerías enlazadas.

Ojo: al hacer esto NetBeans cambia `nbproject/project.properties` con rutas de nuestro equipo. Esos cambios son solo locales, no hay que subirlos a GitHub.

7. Inicialización de la base de datos

El script `sql/init.sql` deja la base de datos lista desde cero. Hace tres cosas: crea la base de datos `biblioteca` si no existe (con `utf8mb4` para que las tildes y la ñ se guarden bien), borra la tabla `libro` si ya existía y la vuelve a crear, e inserta 8 libros de ejemplo. Algunos autores se repiten (George Orwell, Gabriel García Márquez) y hay precios y stocks distintos, para que todas las búsquedas de la aplicación devuelvan algo. Como borra y recrea la tabla, se puede ejecutar todas las veces que se quiera y siempre queda igual.

Atención: el script borra los datos que hubiera en la tabla `libro`. Si hay datos que interese conservar, hay que hacer antes una copia de seguridad (apartado 11).

La tabla que crea es esta:

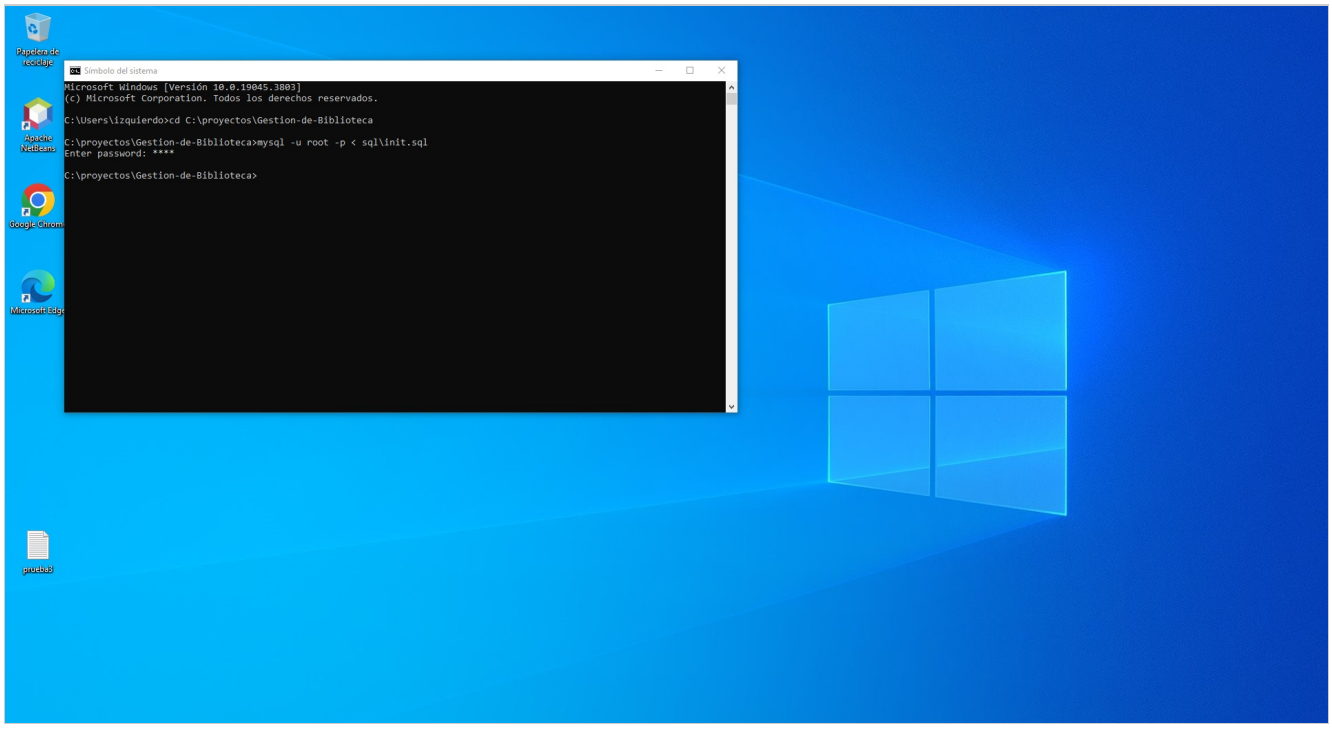
Columna	Tipo	Notas
`idlibro`	INT	Clave primaria, autoincremental
`titulo`	VARCHAR(150)	Obligatorio
`autor`	VARCHAR(100)	Obligatorio
`precio`	DECIMAL(6,2)	Obligatorio, mayor o igual que 0
`stock`	INT	Obligatorio, mayor o igual que 0

7.1. Ejecutar el script

En cmd, desde la raíz del proyecto (pide la contraseña de `root`):

```
cd C:\proyectos\Gestion-de-Biblioteca
mysql -u root -p < sql\init.sql
```

Si va bien no muestra nada y vuelve al prompt.



Captura 16. Ejecución de init.sql en cmd (se ve el comando; si no hay errores no muestra nada más).

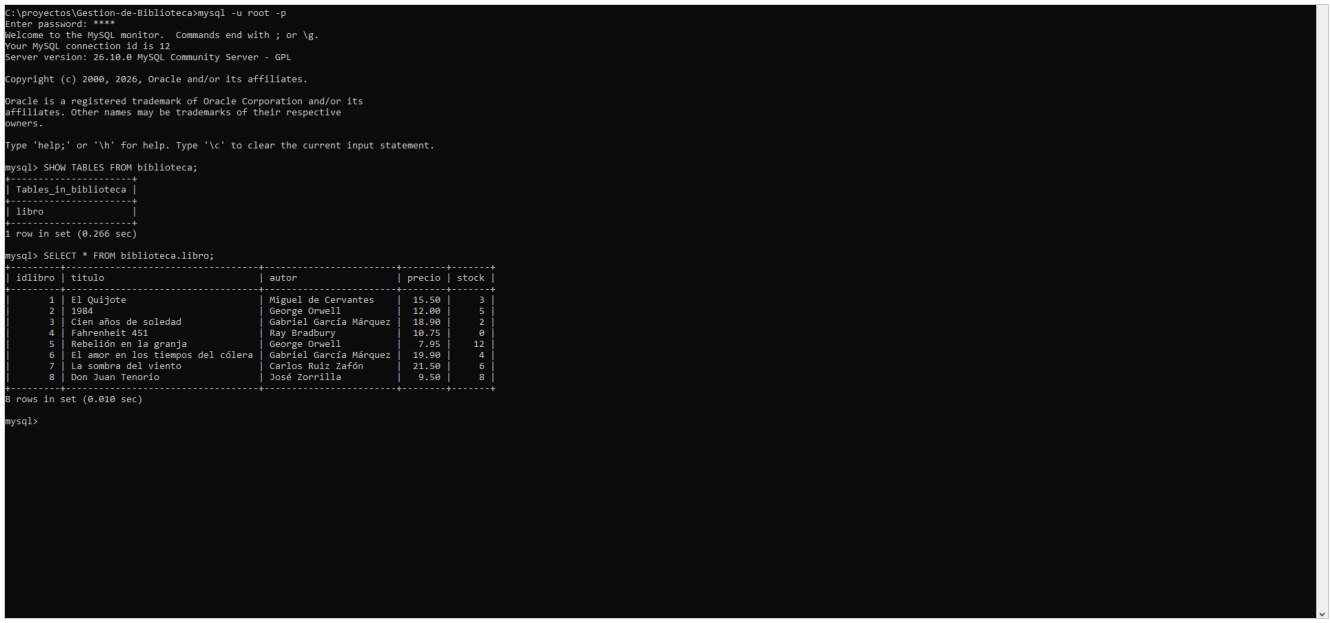
Esto hay que hacerlo en cmd. En PowerShell el operador `<` no funciona y da el error «El operador '<' está reservado para uso futuro» (en inglés, «The '<' operator is reserved for future use»).

Otra forma es entrar en el cliente de MySQL (`mysql -u root -p`) y escribir `source C:/proyectos/Gestion-de-Biblioteca/sql/init.sql`, con barras normales aunque sea Windows.

7.2. Comprobar que ha funcionado

```
mysql -u root -p
mysql> SHOW TABLES FROM biblioteca;
mysql> SELECT * FROM biblioteca.libro;
```

Tiene que salir la tabla `libro` con 8 filas y las tildes bien (por ejemplo «Cien años de soledad»).



Captura 17. SHOW TABLES y SELECT mostrando los 8 libros.

Después lo ejecutamos una segunda vez, para ver que no da error y deja otra vez 8 libros:



Captura 18. Segunda ejecución de init.sql sin errores.

8. Configuración con .env

La aplicación lee los datos de conexión a MySQL del archivo `.env`, no están escritos en el código. La clase `ConexionBD` lo carga con la librería dotenv-java.

Variable	Para qué sirve	Ejemplo	¿Obligatoria?
`DB_URL`	URL JDBC de la base de datos	`jdbc:mysql://localhost:3306/biblioteca`	Sí
`DB_USER`	Usuario de MySQL	`root`	Sí
`DB_PASSWORD`	Contraseña de ese usuario	la de tu MySQL	Sí

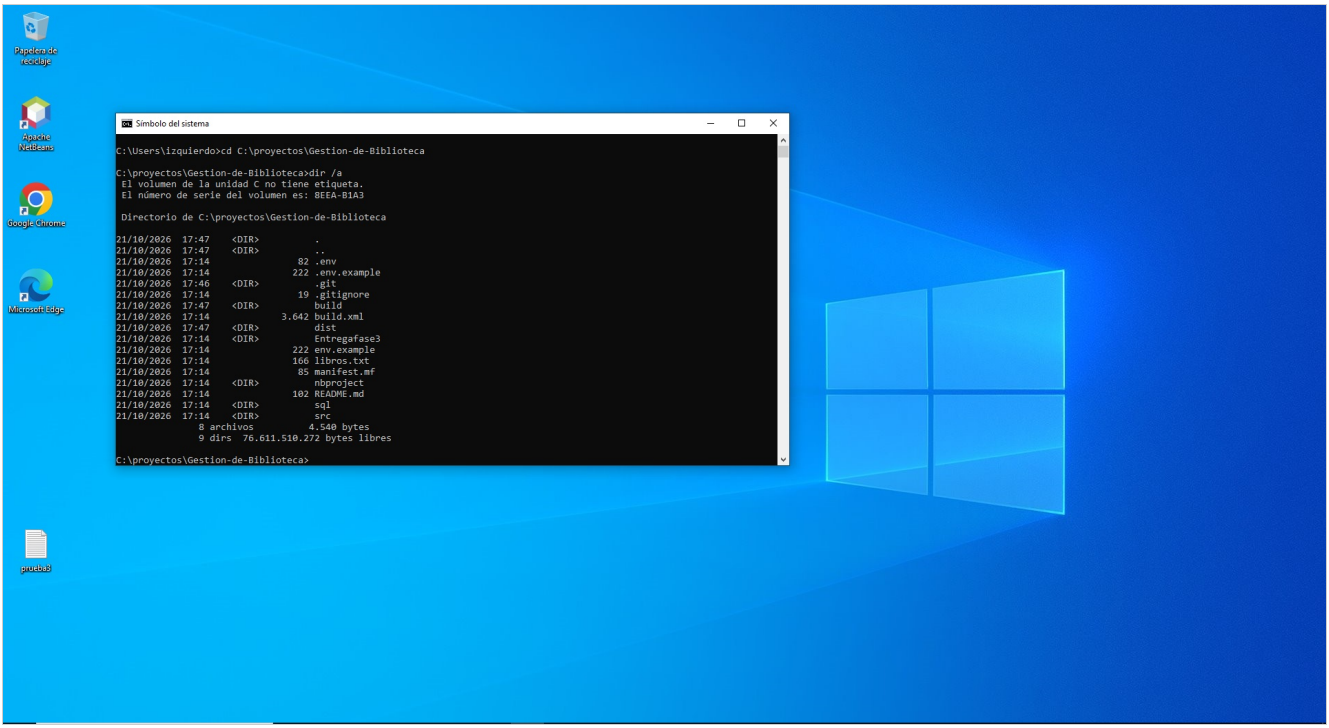
El nombre de la base de datos de `DB_URL` (`biblioteca`) tiene que coincidir con el que crea `init.sql`. La ruta del archivo de texto no está en el `.env`: va fija en el código como `libros.txt`, relativa a la carpeta desde la que se lanza la aplicación.

8.1. Comprobar que el `.env` existe

Al clonar, el repositorio trae ya un `.env`. Se puede ver, en cmd y dentro de la carpeta del proyecto, con:

```
dir /a
```

Tienen que aparecer `.env` y `.env.example`.

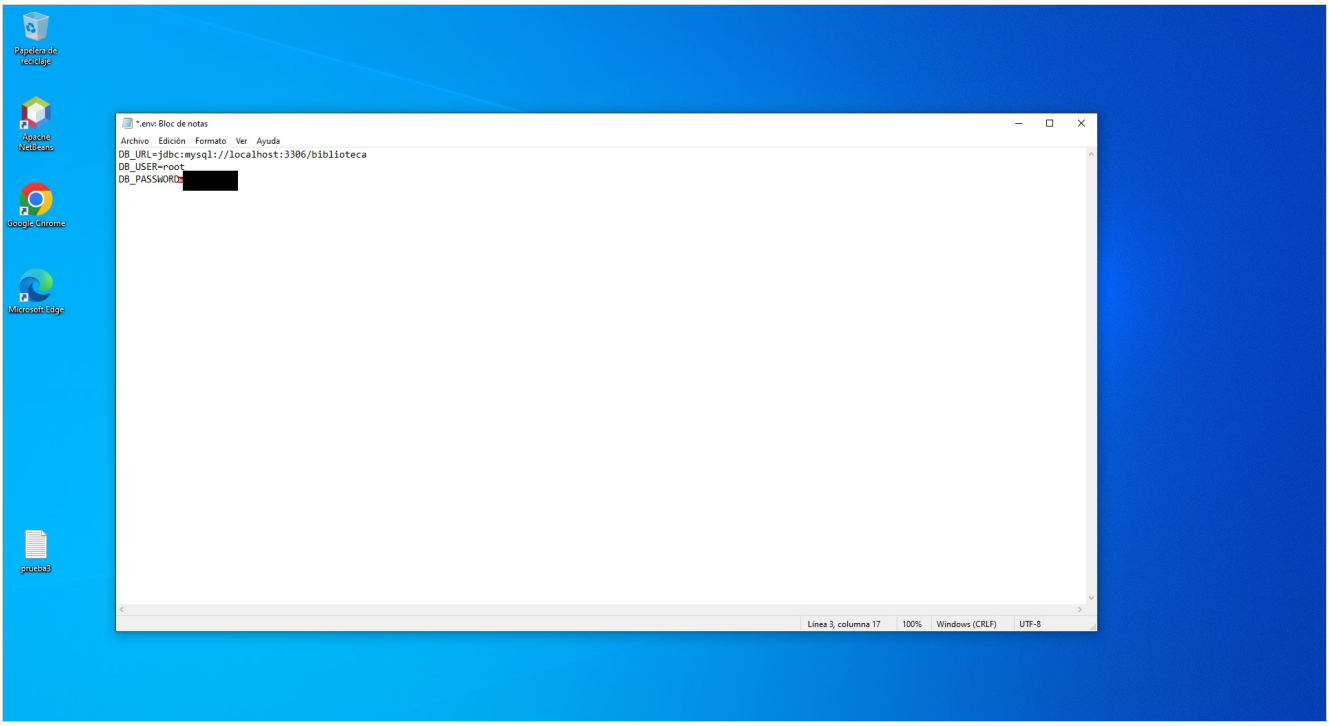


Captura 19. Salida de «dir /a» en la raíz del proyecto, con .env y .env.example visibles.

Si no existiera, se crea a partir de la plantilla con `copy .env.example .env`.

8.2. Editarlo

Se abre con el Bloc de notas (`notepad .env`) y se cambia `DB_PASSWORD` por la contraseña de `root` de nuestro MySQL, la que apuntamos al instalarlo. Si se usa otro usuario que no sea `root`, también hay que cambiar `DB_USER`. `DB_URL` se deja igual si la base de datos se llama `biblioteca`.



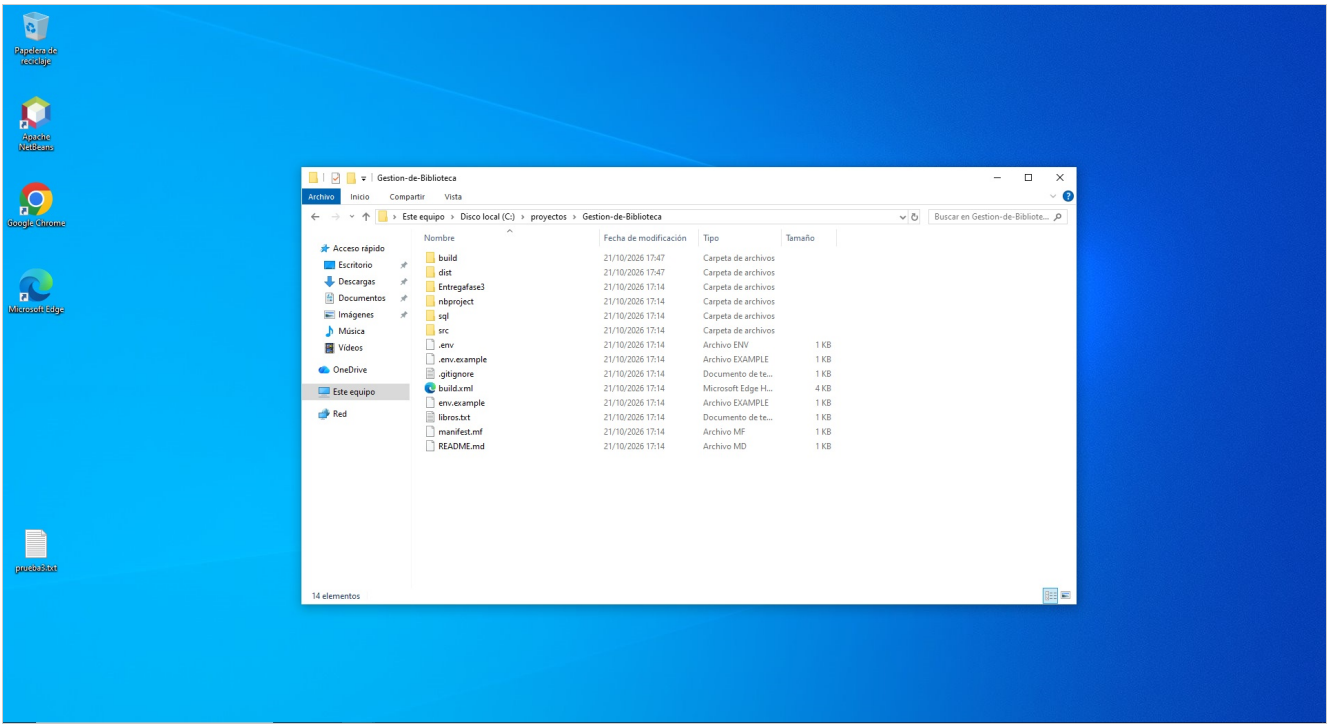
Captura 20. Archivo .env abierto en el editor (con la contraseña tapada).

El `.env` tiene que estar en la raíz del proyecto, que es la carpeta desde la que se lanza la aplicación (apartado 9).

8.3. Cuidado con las extensiones

Con las extensiones ocultas el Bloc de notas puede guardar el archivo como `.env.txt` sin que se note, y entonces la aplicación no lo encuentra. Para evitarlo, en el Explorador se activa Vista → Mostrar → Extensiones de nombre de archivo

y se comprueba que se llama exactamente `.env`.

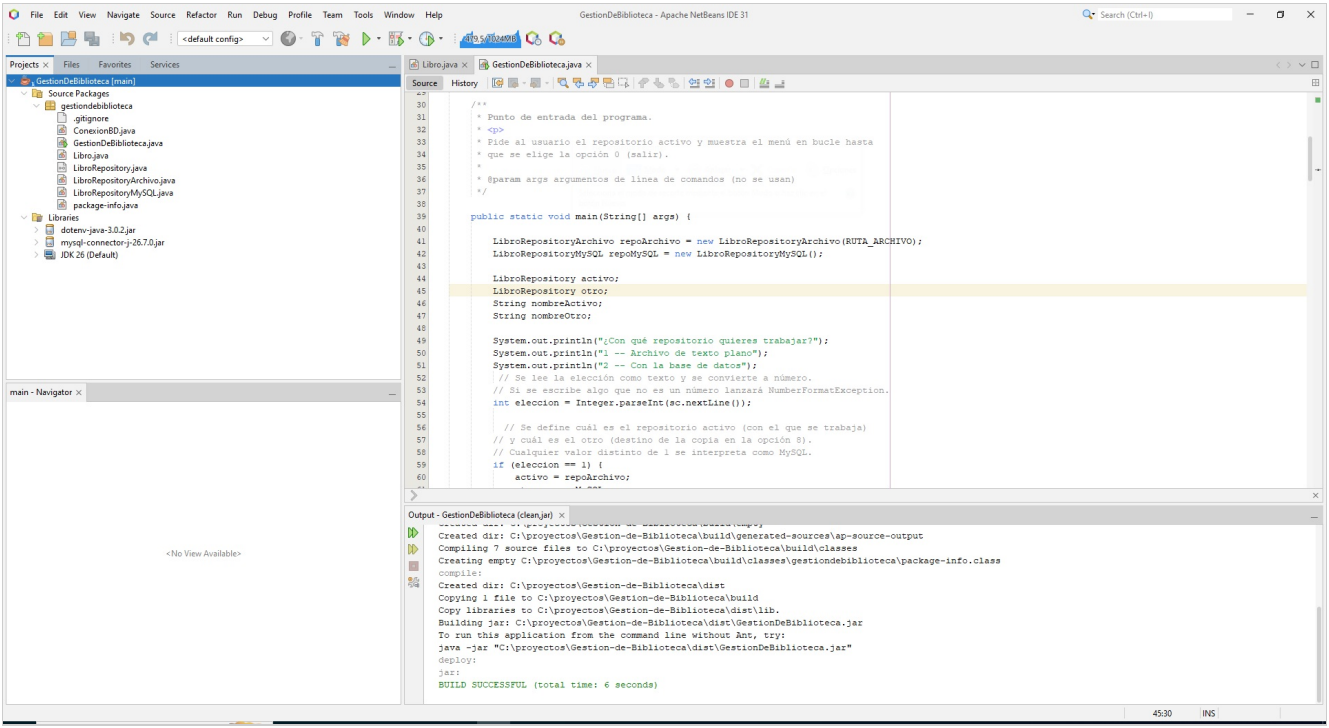


Captura 21. Explorador de archivos mostrando «.env» con las extensiones visibles.

9. Arranque de la aplicación

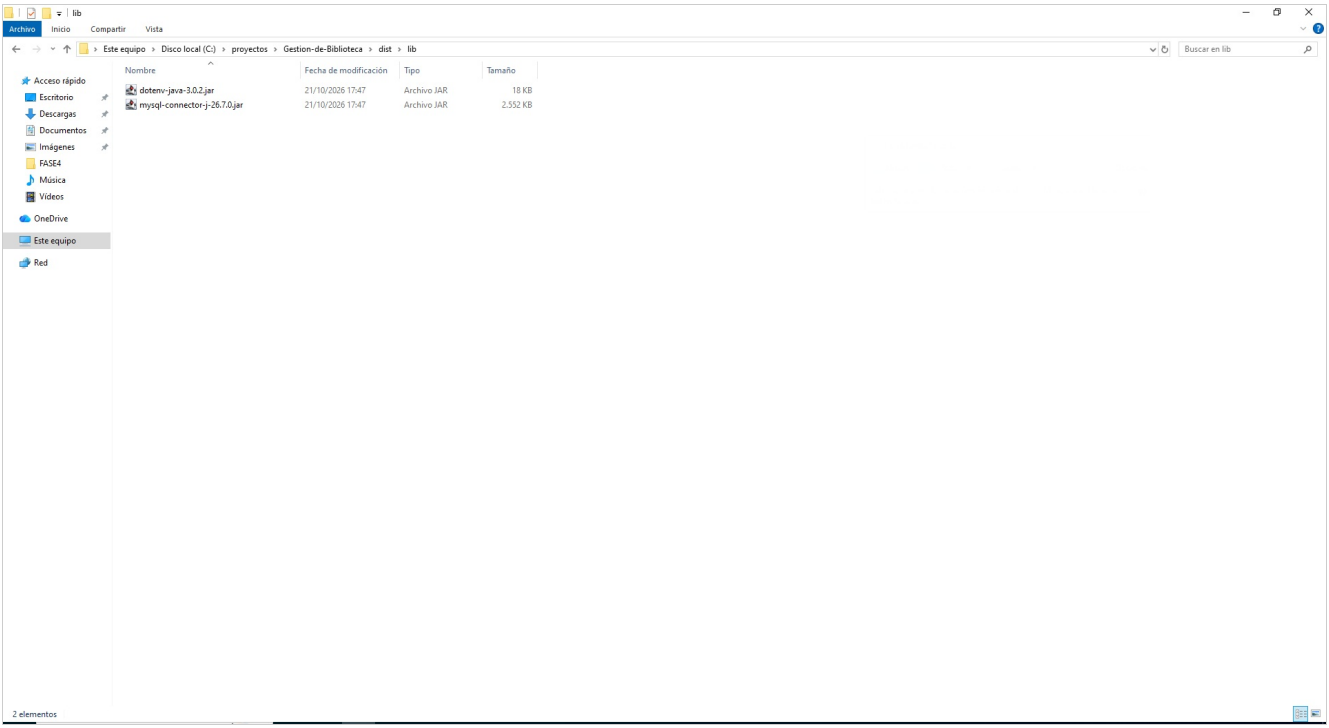
9.1. Desde NetBeans

Clic derecho sobre el proyecto → Clean and Build. Tiene que acabar en `BUILD SUCCESSFUL`.



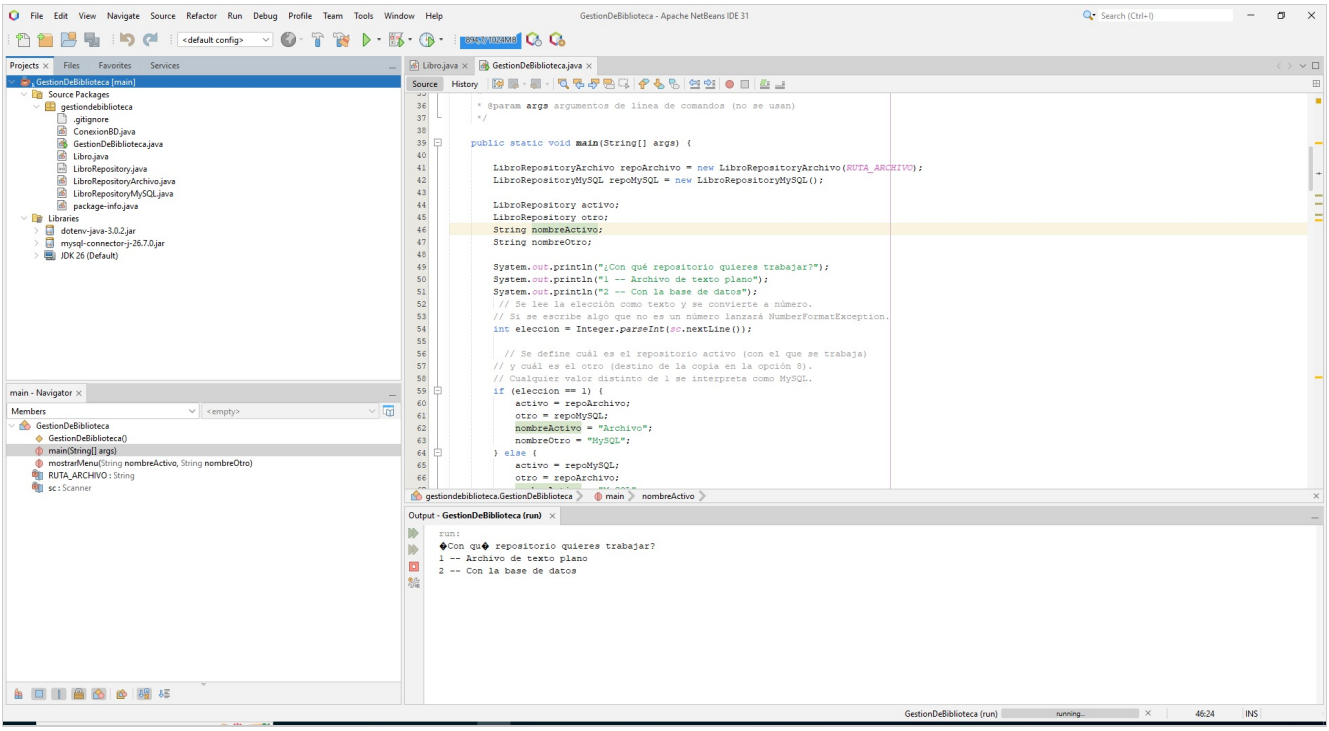
Captura 22. Clean and Build terminado con BUILD SUCCESSFUL.

Con eso se crea la carpeta `dist/` con `GestionDeBiblioteca.jar` y, dentro, `dist/lib/` con las dos librerías.



Captura 23. Carpeta distcon las dos librerías copiadas por Clean and Build.

La clase principal ya viene configurada (Properties → Run → Main Class: `gestiondebiblioteca.GestionDeBiblioteca`). Pulsando Run aparece la pregunta «¿Con qué repositorio quieres trabajar?». NetBeans usa la raíz del proyecto como carpeta de trabajo, y ahí tienen que estar el `.env` y `libros.txt`.

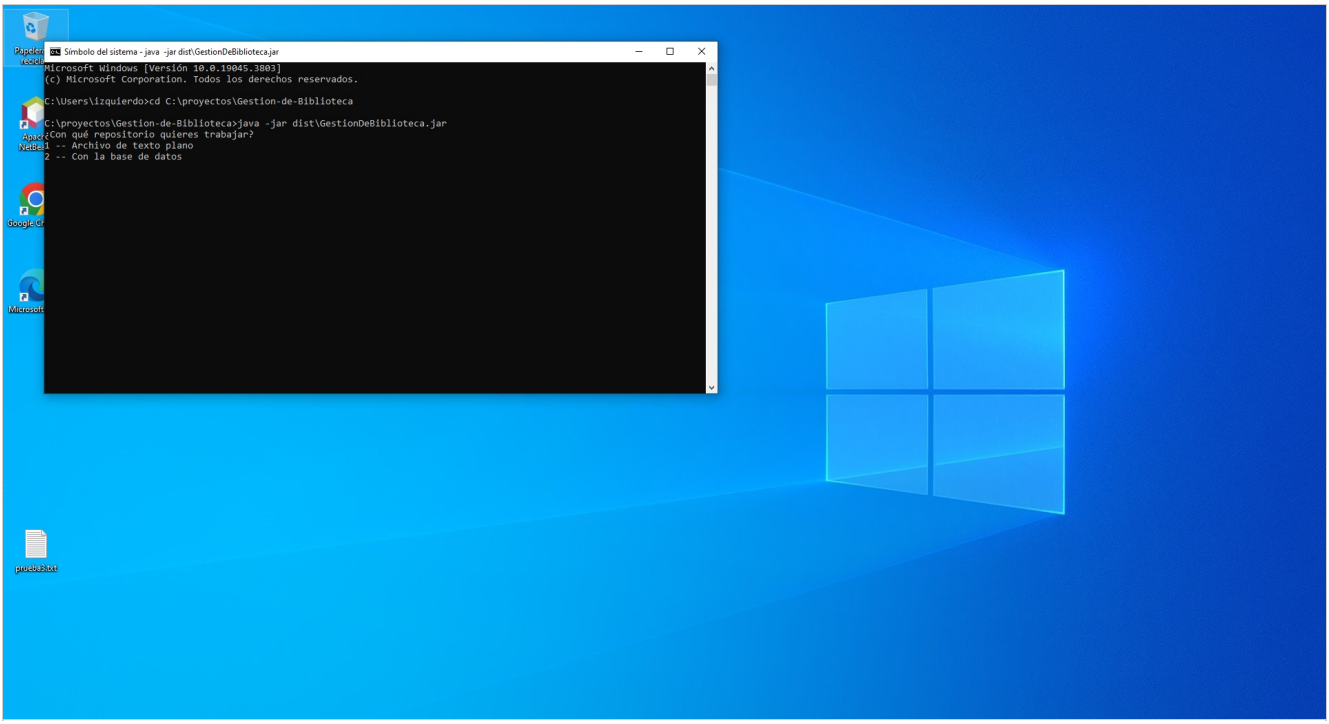


Captura 24. Aplicación arrancada desde NetBeans, con el menú inicial visible.

9.2. Desde la terminal

Después del Clean and Build, en cmd y desde la raíz del proyecto:

```
cd C:\proyectos\Gestion-de-Biblioteca
java -jar dist\GestionDeBiblioteca.jar
```



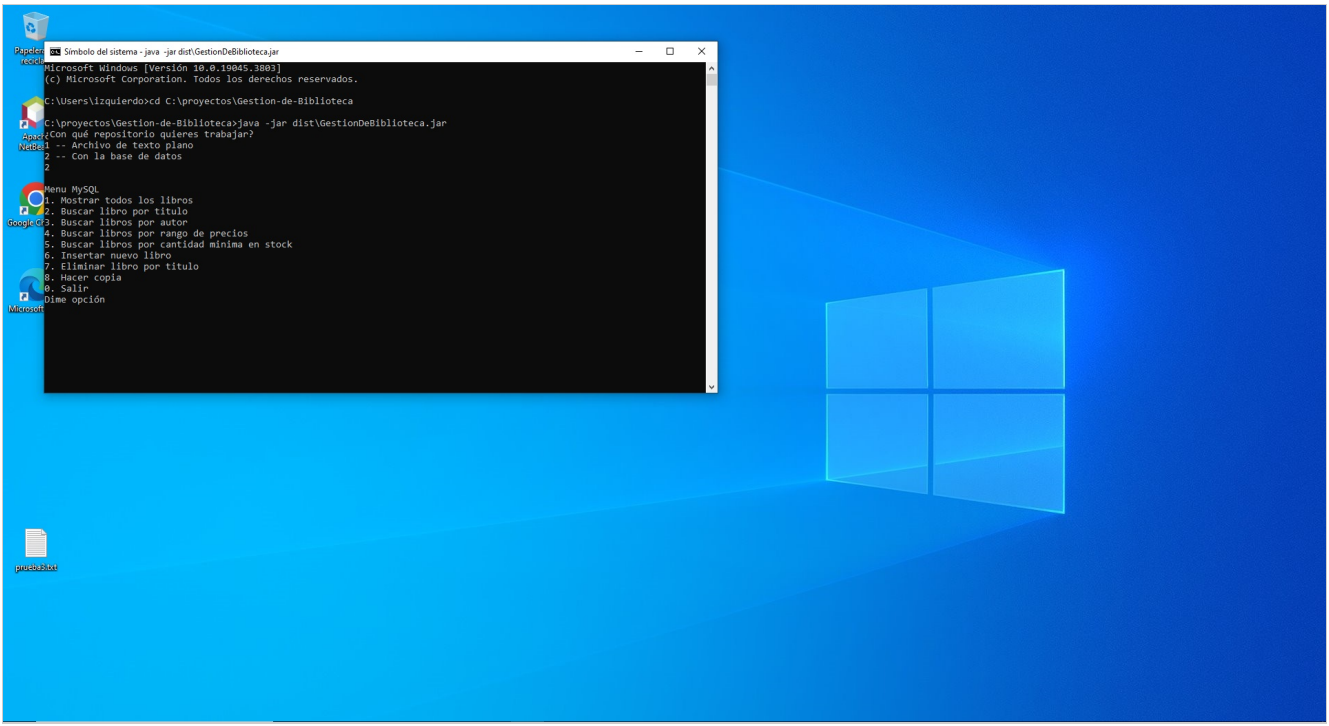
Captura 25. Aplicación arrancada desde cmd, con el menú inicial visible.

Hay tres cosas que tener en cuenta. Hay que lanzarlo siempre desde la raíz del proyecto y no desde dentro de `dist/`, porque si no, no encuentra el `.env` y además crea un `libros.txt` vacío en la carpeta equivocada. Si se mueve el `.jar` a otro sitio hay que mover la carpeta `dist/` entera, porque busca las librerías en `dist/lib/`. Y hace falta Java 26 o superior.

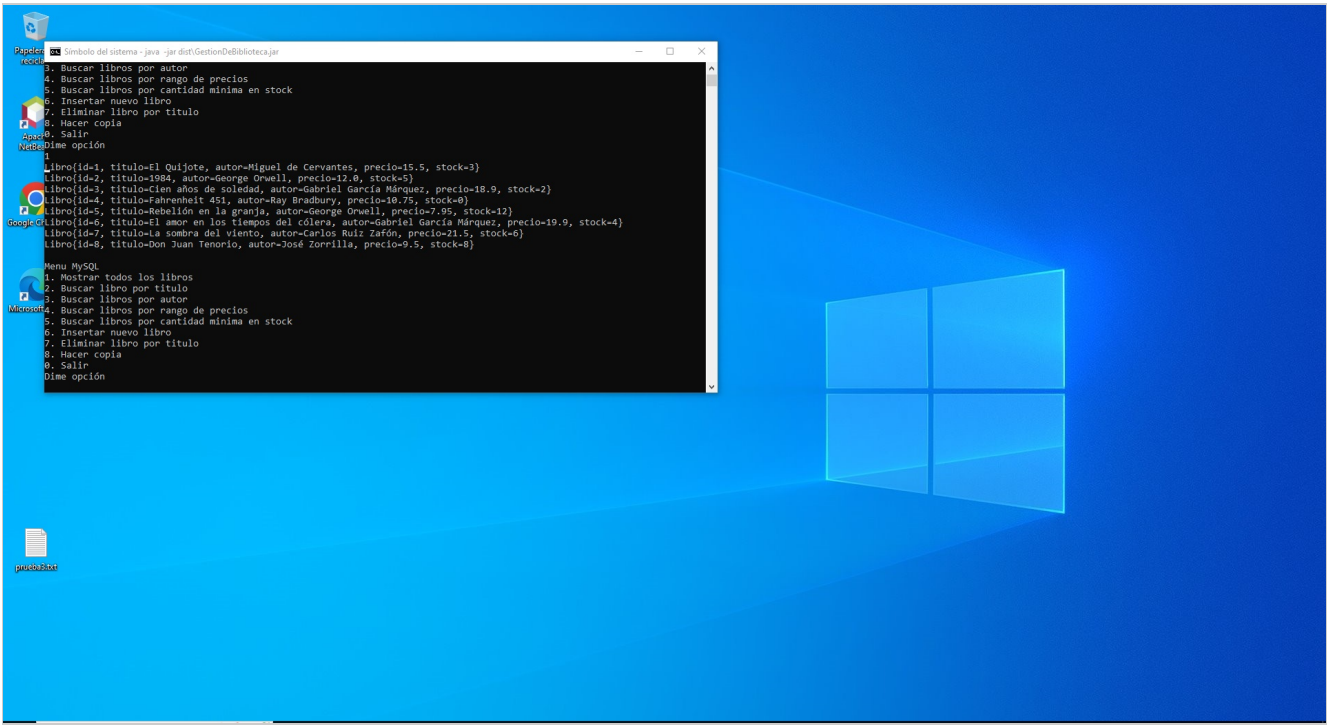
10. Comprobación del despliegue

Con que arranque no basta, así que hacemos esta secuencia con los dos repositorios. Al arrancar la aplicación pregunta: 1 es archivo de texto y 2 es base de datos.

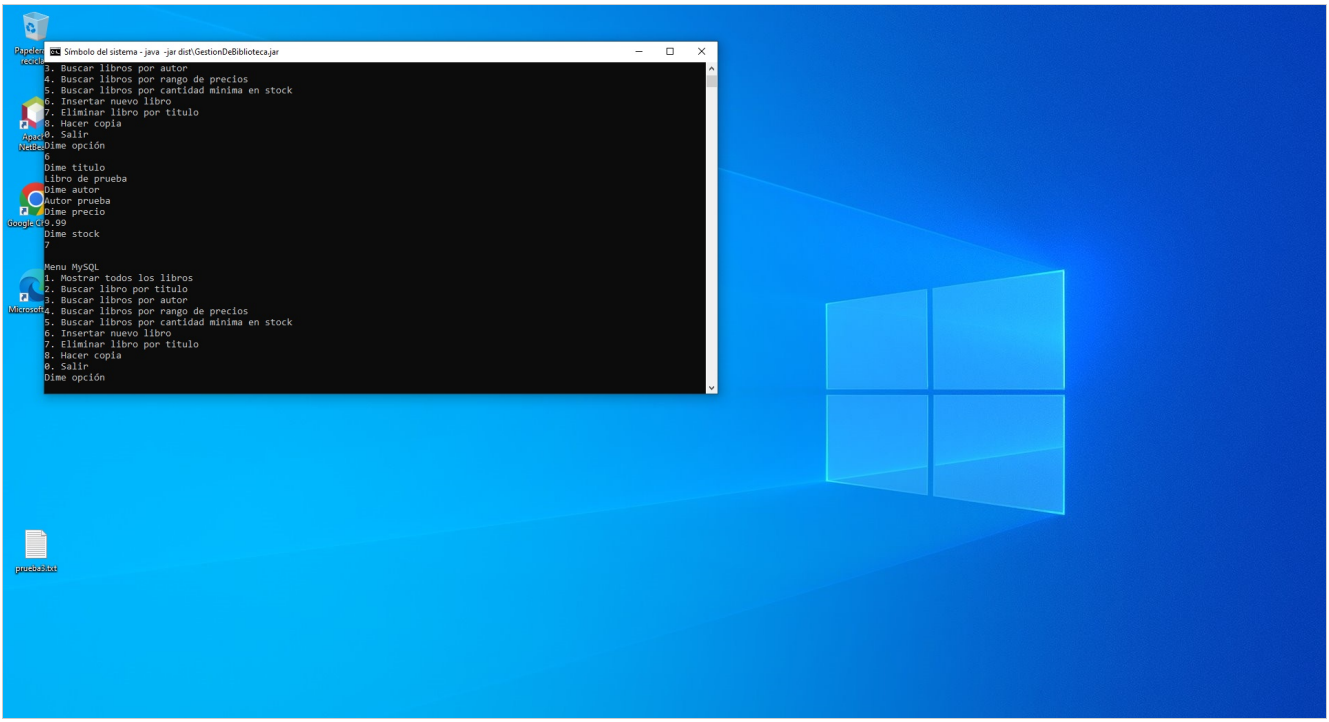
Nº	Acción	Resultado esperado
1	Arrancar y elegir 2 (MySQL)	Aparece el «Menu MySQL» sin errores de conexión
2	Opción 1 (mostrar todos)	Salen los 8 libros de `init.sql`
3	Opción 6: insertar un libro de prueba (título, autor, precio, stock)	Vuelve al menú. La aplicación no muestra mensaje de confirmación
4	En MySQL: `SELECT * FROM biblioteca.libro;`	El libro nuevo aparece en la tabla
5	Salir con 0, arrancar de nuevo y elegir 1 (archivo)	Aparece el «Menu Archivo»
6	Opción 1 (mostrar todos)	Salen los libros de `libros.txt` (4 en el archivo original)
7	Opción 6: insertar un libro y abrir `libros.txt`	El libro aparece como línea nueva al final, con formato `id^titulo^autor^precio^stock`



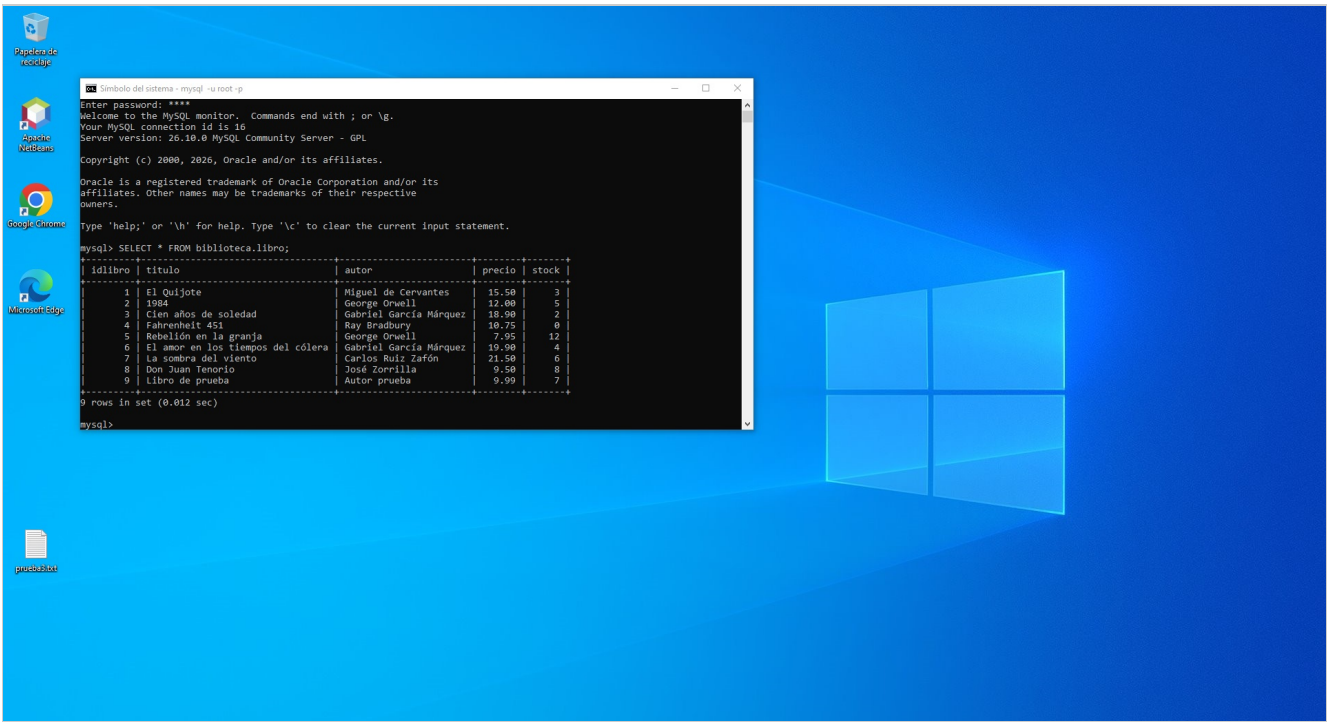
Captura 26. Fila 1: arranque eligiendo MySQL, menú visible.



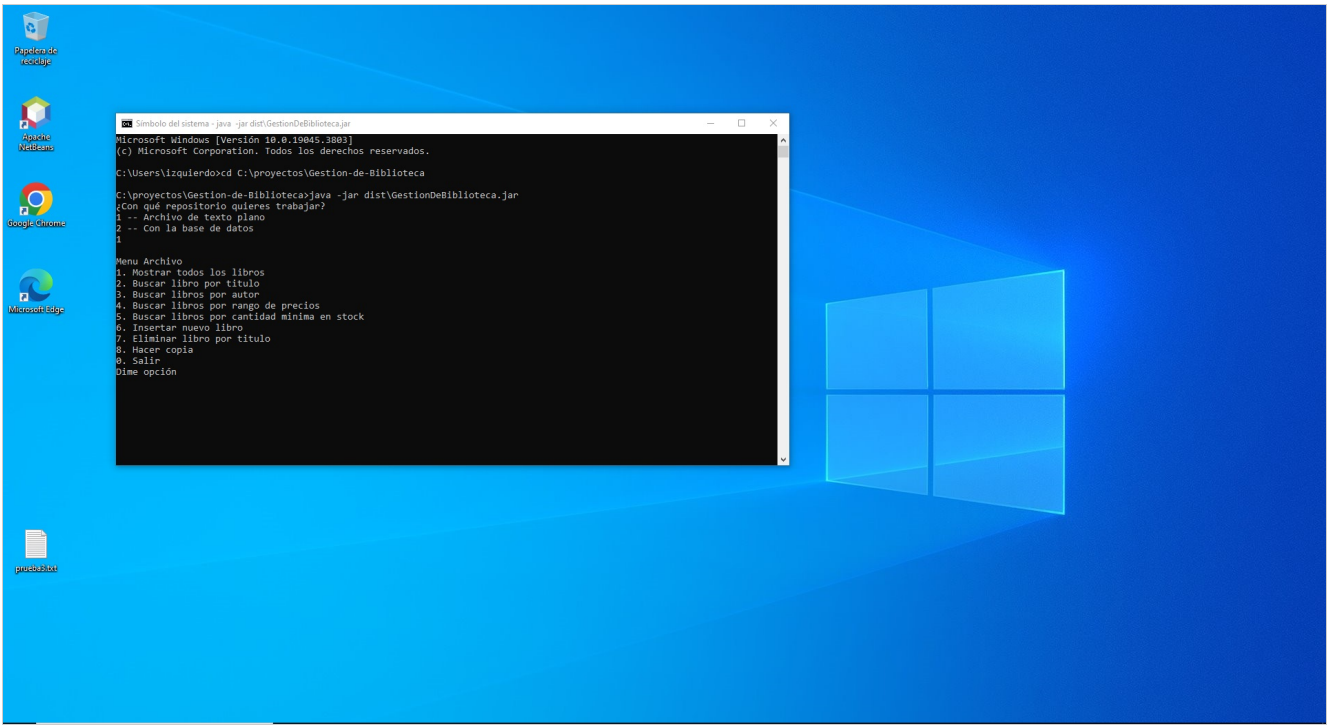
Captura 27. Fila 2: listado con los 8 libros de init.sql.



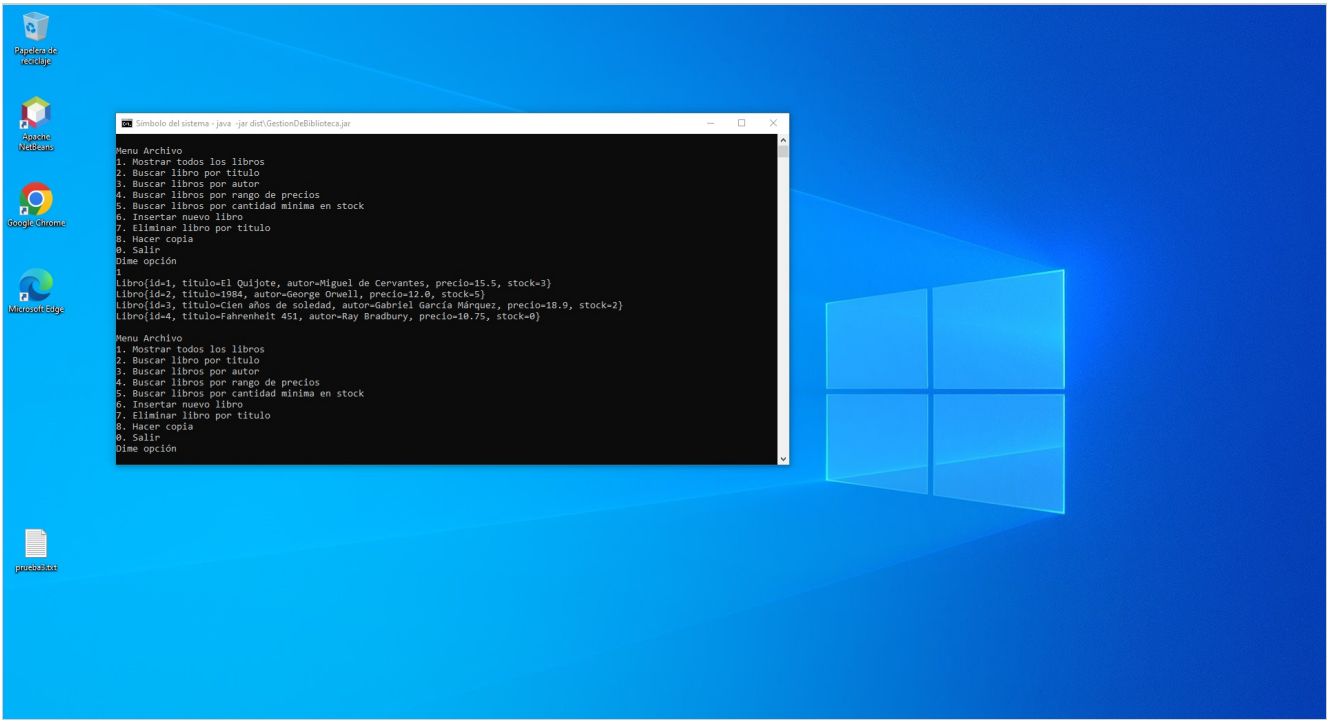
Captura 28. Fila 3: inserción de un libro de prueba.



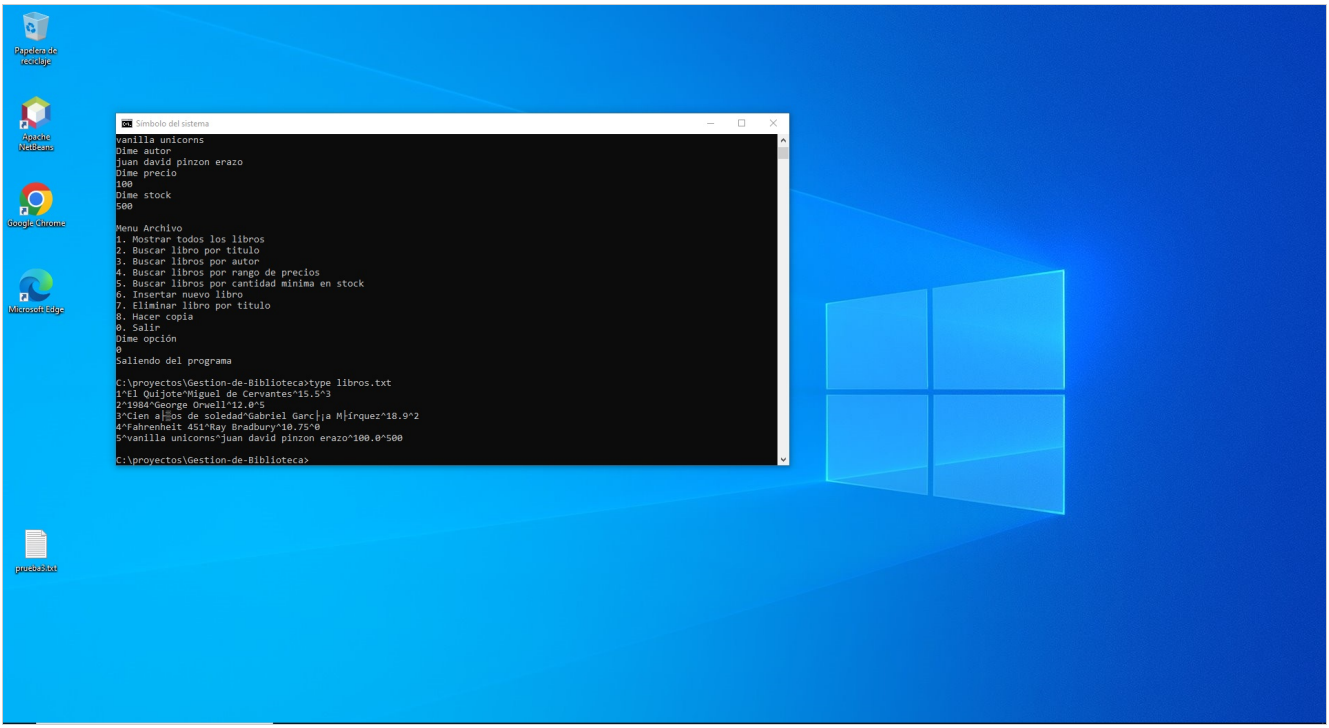
Captura 29. Fila 4: el libro nuevo visible con SELECT en MySQL.



Captura 30. Fila 5: arranque eligiendo archivo, menú visible.



Captura 31. Fila 6: listado con los libros de libros.txt.



Captura 32. Fila 7: libro insertado visible en libros.txt.

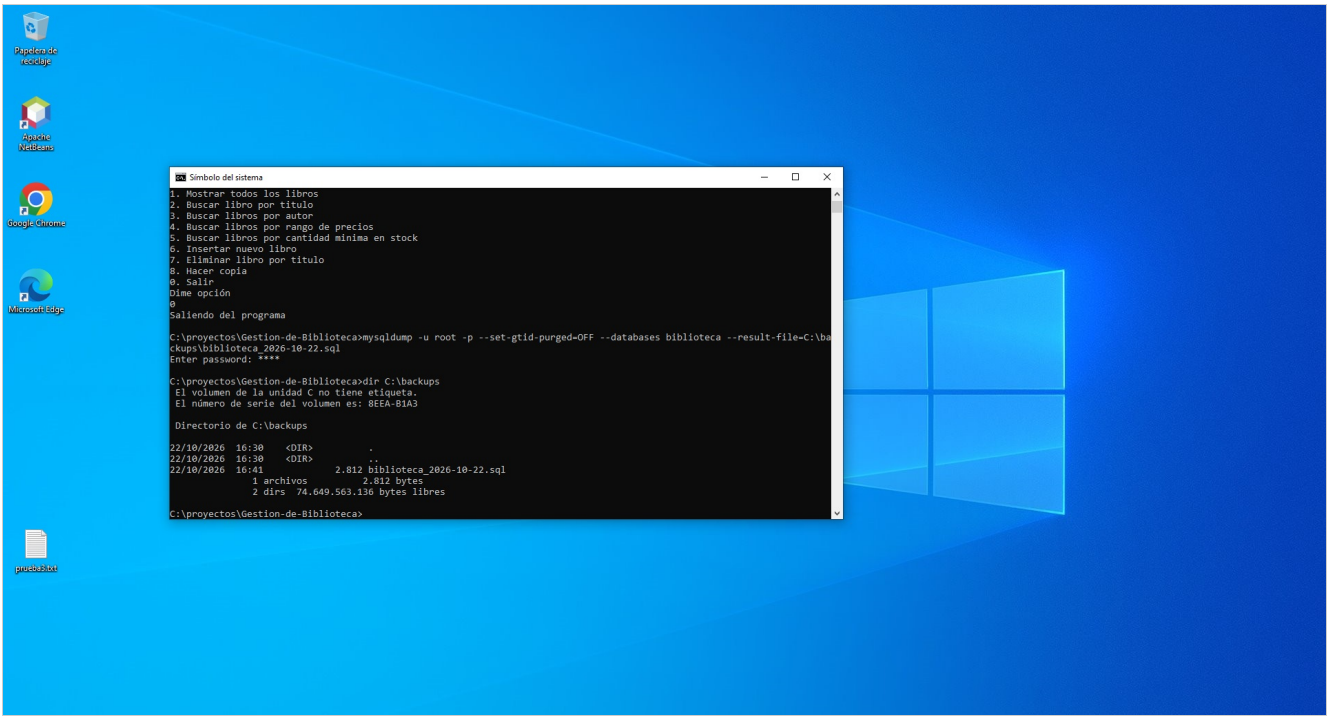
11. Administración

Tareas de mantenimiento una vez instalada la aplicación. Todo en cmd.

11.1. Copia de seguridad de la base de datos

```
mkdir C:\backups
mysqldump -u root -p --set-gtid-purged=OFF --databases biblioteca --result-file=C:\backups\biblioteca_2026-10-22.sql
```

`--databases` hace que el volcado incluya también la creación de la base de datos, para poder restaurarlo en un servidor vacío. `--set-gtid-purged=OFF` evita que meta sentencias de GTID, así la restauración no depende de la configuración de replicación del servidor. Usamos `--result-file` y no `> archivo.sql` porque en PowerShell el `>` guarda el archivo en UTF-16 y luego no se puede restaurar con `mysql`.

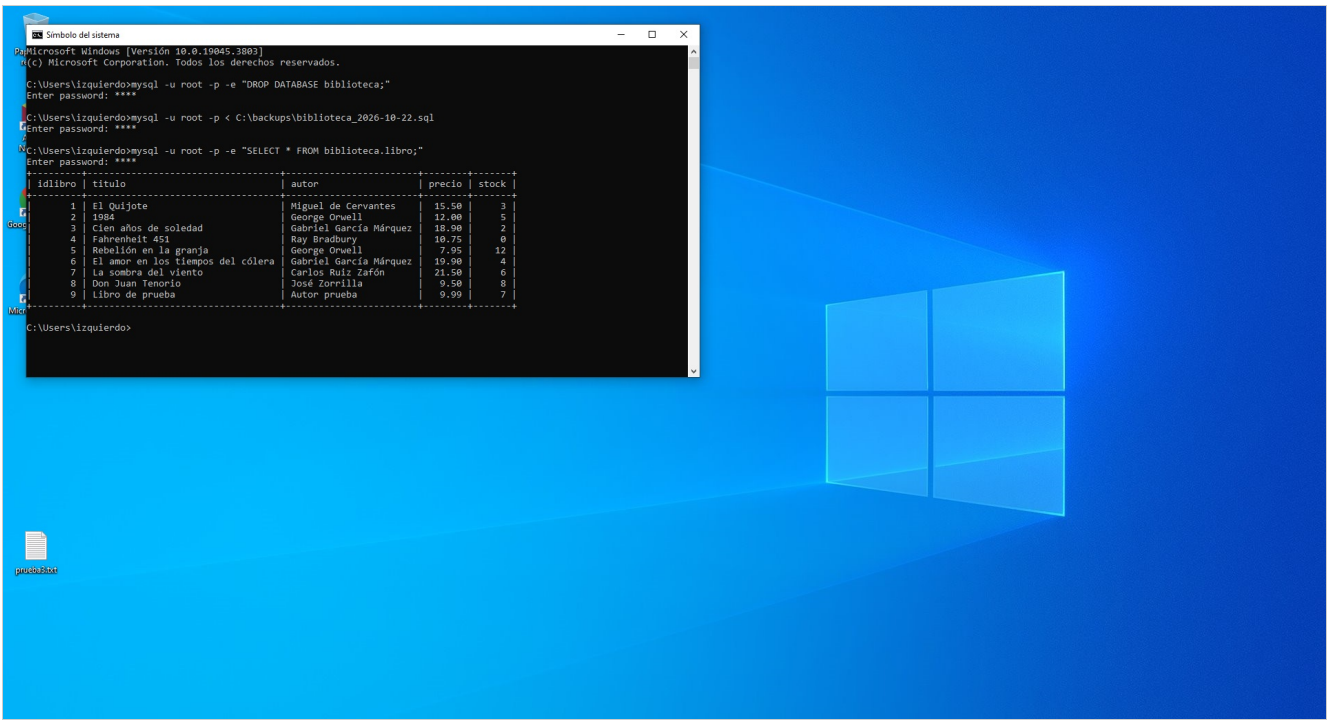


Captura 33. Copia de seguridad con mysqldump y el archivo .sql resultante.

11.2. Restaurar una copia de seguridad

```
mysql -u root -p < C:\backups\biblioteca_2026-10-22.sql
```

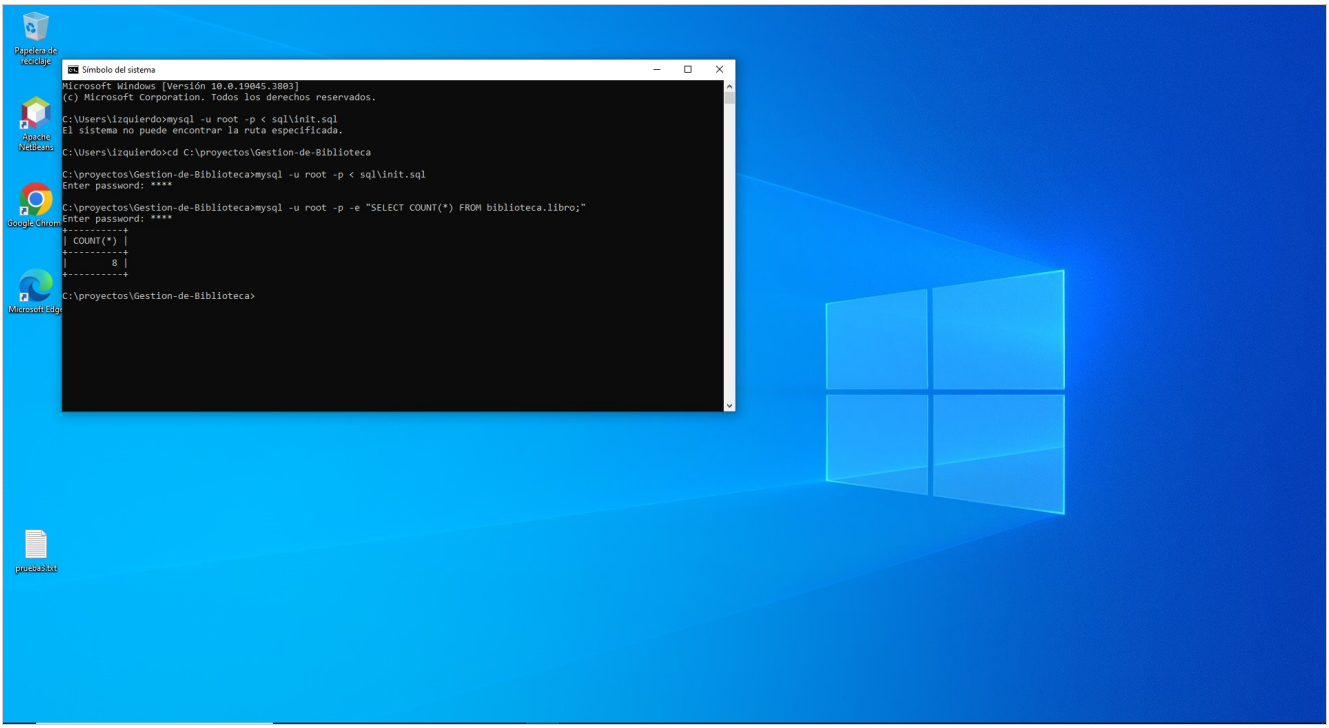
También se puede con `source` desde el cliente de MySQL, como en el apartado 7. Nosotros primero borramos la base de datos (`DROP DATABASE biblioteca;`), restauramos y comprobamos con un `SELECT` que volvían los libros, incluido el de prueba que habíamos insertado.



Captura 34. Restauración de la copia y comprobación con SELECT.

11.3. Volver a los datos iniciales

Se ejecuta otra vez `init.sql` (apartado 7.1). Se pierden todos los datos actuales de la tabla `libro`.



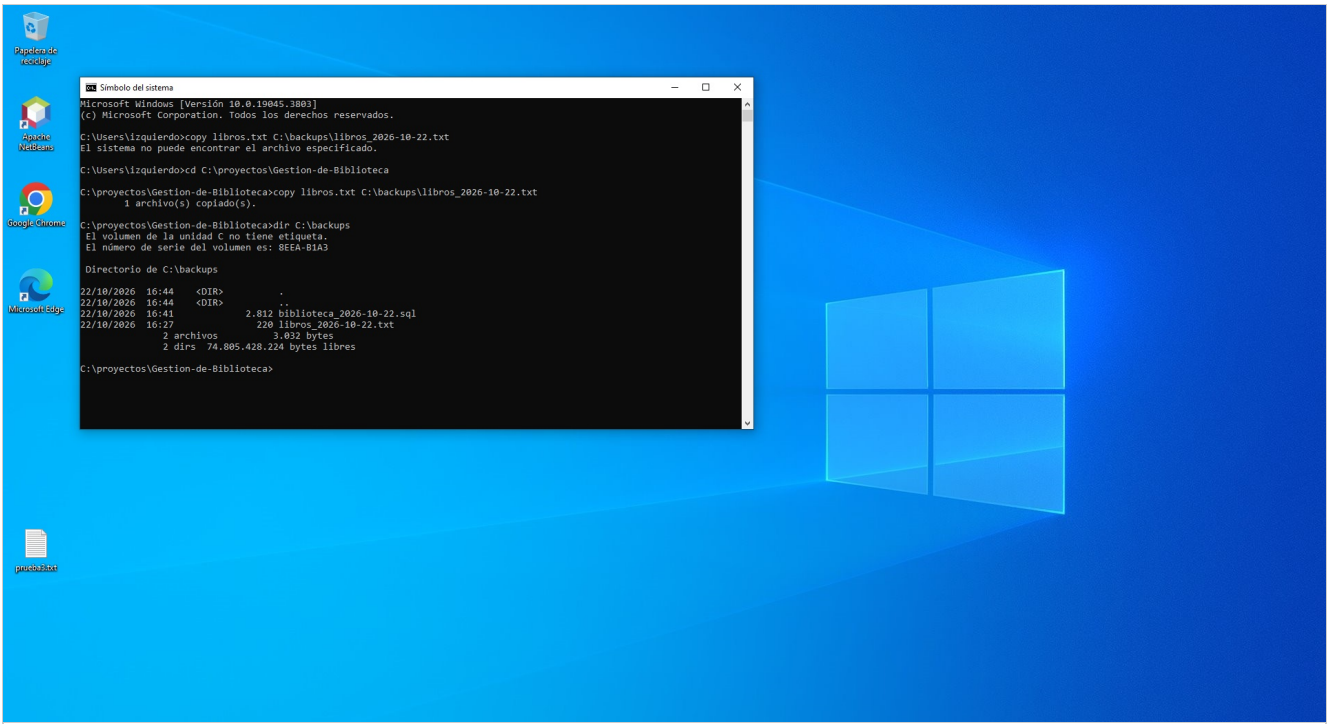
Captura 35. Ejecución de init.sql para volver a los 8 libros iniciales.

11.4. Copia de seguridad del archivo de texto

Los datos del repositorio de archivo están en `libros.txt`, en la raíz del proyecto (o en la carpeta desde la que se lance la aplicación). Se copia como cualquier archivo:

```
copy libros.txt C:\backups\libros_2026-10-22.txt
```

Para restaurarlo se copia de vuelta con el nombre `libros.txt`.



Captura 36. Copia de libros.txt a C:.

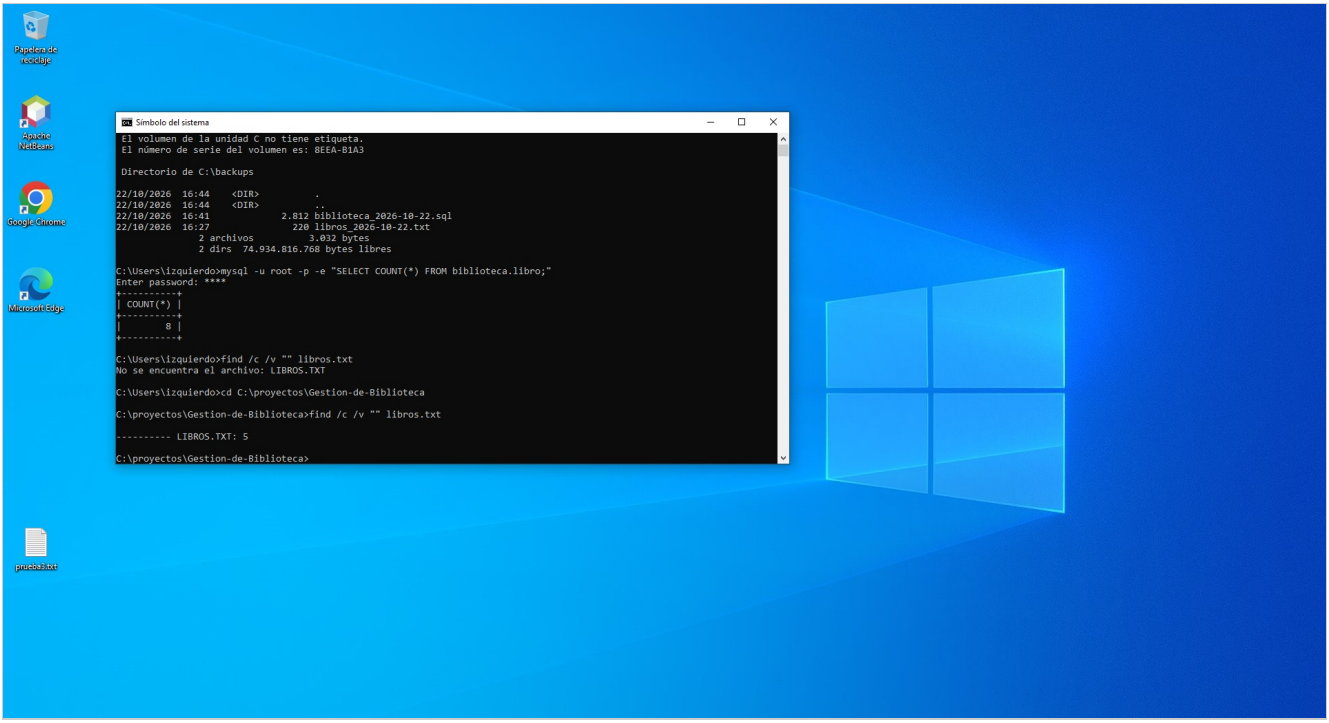
11.5. Copiar datos entre repositorios (opción 8)

La opción 8, «Hacer copia», copia todos los libros del repositorio con el que estamos trabajando al otro: del archivo a MySQL si trabajamos con el archivo, y de MySQL al archivo si trabajamos con MySQL.

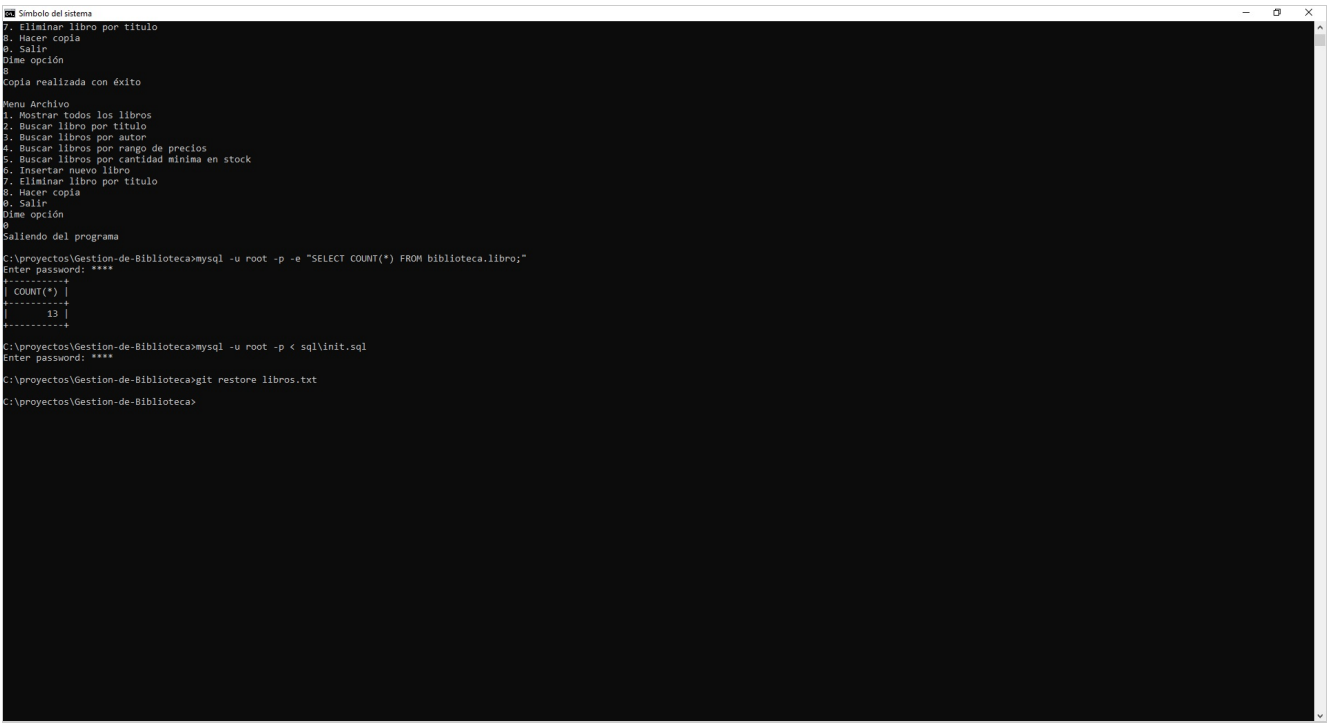
Lo que comprobamos es que no detecta duplicados ni sobrescribe nada: cada libro se inserta como uno nuevo en el destino, con un identificador nuevo (el que asigna MySQL, o el siguiente libre en el archivo). Esto significa que si el destino ya tenía los mismos libros quedan duplicados, que si se pulsa dos veces se duplican dos veces, y que los identificadores del origen no se conservan.

Por ejemplo, con 8 libros en MySQL y 5 líneas en `libros.txt`, la opción 8 trabajando con el archivo dejó 13 libros en MySQL (8 más 5). En el otro sentido pasa igual: todos los libros se añaden al final de `libros.txt`. Cuando termina bien muestra «Copia realizada con éxito», y si el origen está vacío o falla algún libro muestra «Hubo algún problema al copiar, o no había libros que copiar».

Por eso lo más sensato es usarla solo para volcar datos a un repositorio vacío.



Captura 37. Antes de la copia: número de libros en el destino.



Captura 38. Después de la copia (opción 8): libros duplicados en el destino.

11.6. Actualizar a una versión nueva

```
cd C:\proyectos\Gestion-de-Biblioteca
git pull
```

Después se vuelve a hacer Clean and Build en NetBeans. El `.env` no se pierde, pero ojo: como el repositorio también lo trae, si cambia en GitHub Git puede avisar de un conflicto con nuestra copia local.

11.7. Cambiar de base de datos o de usuario

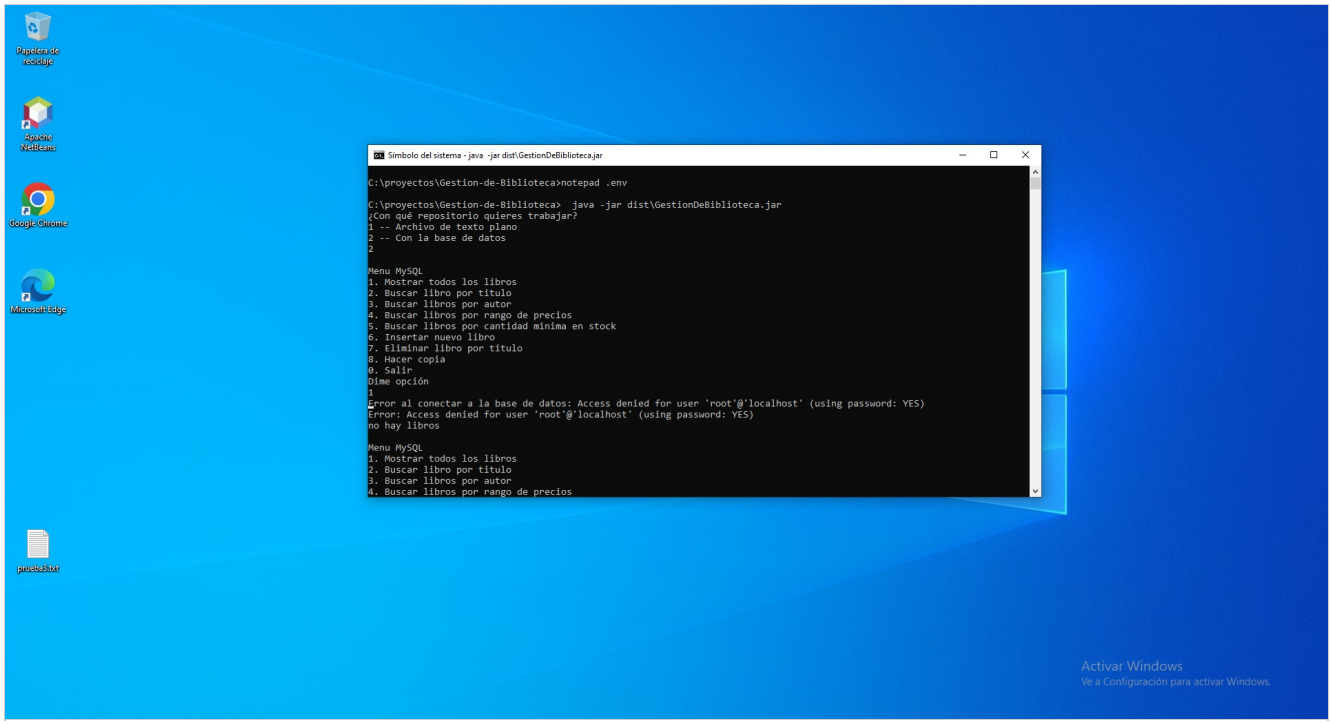
Se edita el `.env` (apartado 8): `DB_URL` para otra base de datos o servidor, y `DB_USER` y `DB_PASSWORD` para otro usuario. No hace falta recompilar, basta con volver a arrancar la aplicación.

12. Resolución de problemas

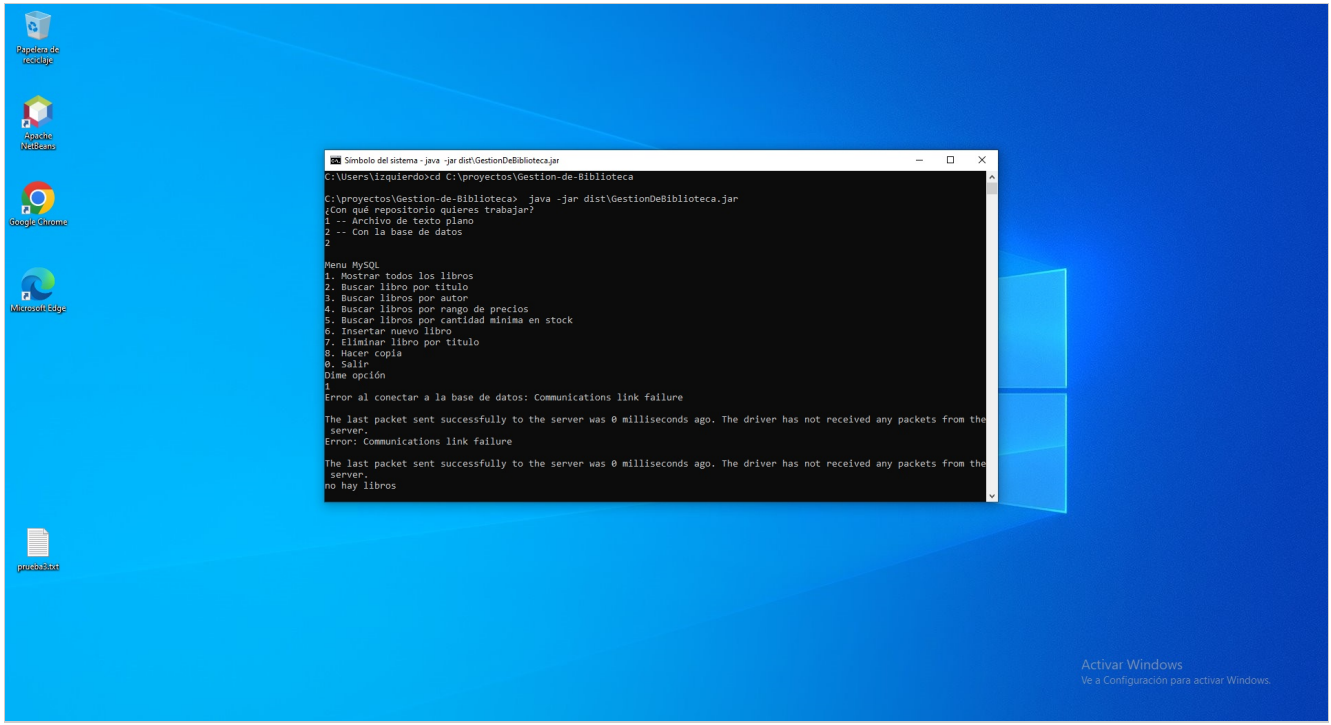
Estos errores los provocamos a propósito en la máquina virtual, y los mensajes son los que muestra la aplicación.

Error	Causa	Solución
`Access denied for user 'root'@'localhost' (using password: YES)`	Usuario o contraseña del `.env` incorrectos.	Revisar `DB_USER` y `DB_PASSWORD` en el `.env`.
`Communications link failure`	El servidor MySQL no está arrancado.	Iniciar el servicio de MySQL en `services.msc`.
`Unknown database 'biblioteca'`	No se ha ejecutado `init.sql`, o el nombre de la base de datos en `DB_URL` no coincide.	Ejecutar `init.sql` (apartado 7) y revisar `DB_URL`.
`No suitable driver found for jdbc:mysql://localhost:3306/biblioteca`	Falta el conector MySQL.	Comprobar que `mysql-connector-j-26.7.0.jar` está enlazado en Properties → Libraries (apartado 6) y repetir Clean and Build.
`NoClassDefFoundError: io/github/cdimascio/dotenv/Dotenv`	Falta la librería dotenv-java, o se ha movido el `.jar` sin su carpeta `lib`.	Revisar Properties → Libraries (apartado 6) y repetir Clean and Build. Si se ejecuta el `.jar`, tiene que estar junto a `dist/lib/`.
`ExceptionInInitializerError` con `DotenvException: Could not find /.env on the classpath`	No hay `.env` en la carpeta de trabajo (por ejemplo, se lanzó el `.jar` desde dentro de `dist`).	Comprobar que existe el `.env` y lanzar desde la raíz del proyecto.
`UnsupportedClassVersionError: ... class file version 70.0, this version of the Java Runtime only recognizes class file versions up to ...`	El Java instalado es anterior al 26.	Instalar el JDK 26 y comprobar con `java -version` (apartado 3.2).
`NumberFormatException: For input string: "hola"`	Se ha escrito texto donde se esperaba un número (opción de menú, precio o stock). La aplicación se cierra.	Volver a arrancar y escribir solo números donde se piden.
Sale «no hay libros» justo después de un error de conexión	La aplicación enseña una lista vacía cuando falla la base de datos. No significa que la tabla esté vacía.	Leer el error de conexión que sale antes y resolverlo (filas anteriores).
`error: package io.github.cdimascio.dotenv does not exist` y `BUILD FAILED`	El proyecto guarda las rutas de las librerías de otro ordenador y no las encuentra.	Descargar los `.jar` y enlazarlos como en el apartado 6.
`"mysql" no se reconoce como un comando interno o externo, programa o archivo por lotes ejecutable`	MySQL no está en el PATH.	Añadir la carpeta `bin` de MySQL al PATH y abrir una cmd nueva (apartado 3.3).
`El operador '<' está reservado para uso futuro` (en inglés, `The '<' operator is reserved for future use`)	Se ha usado PowerShell para `mysql ... < sql\init.sql`.	Usar cmd, o `source` desde el cliente de MySQL (apartado 7).

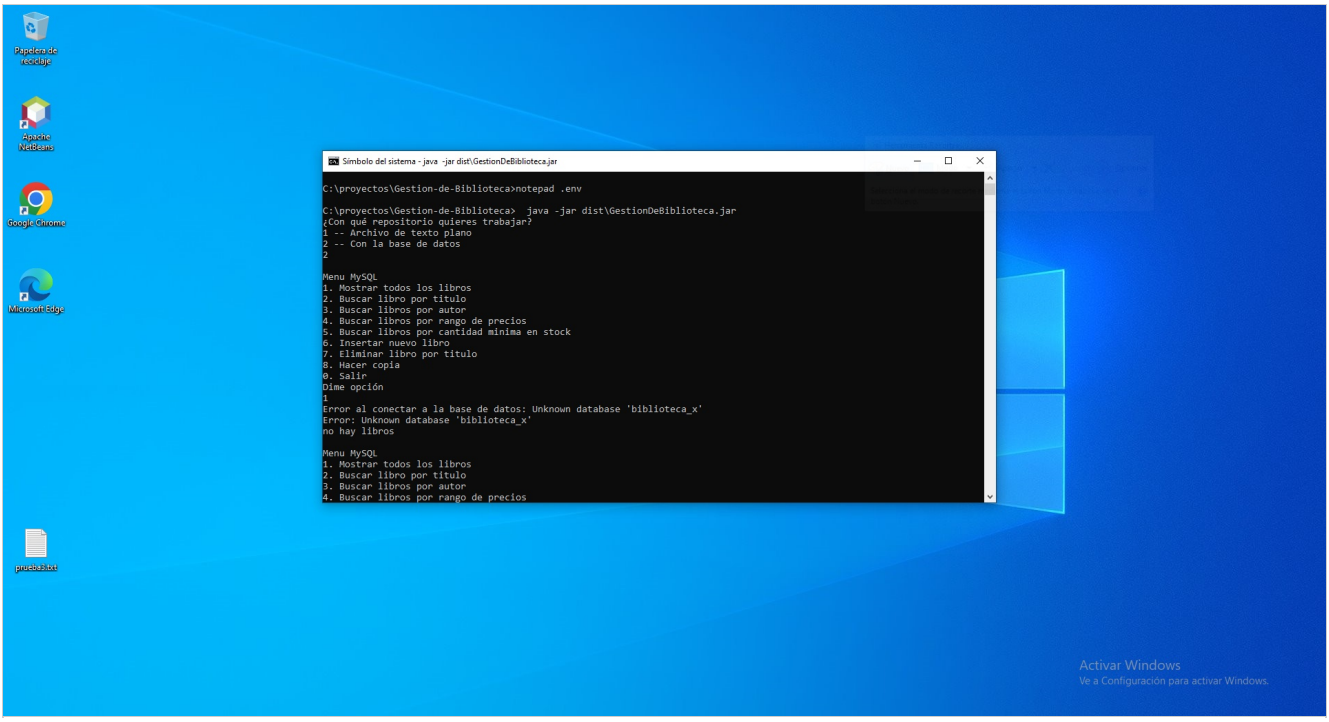
Las capturas de cada uno:



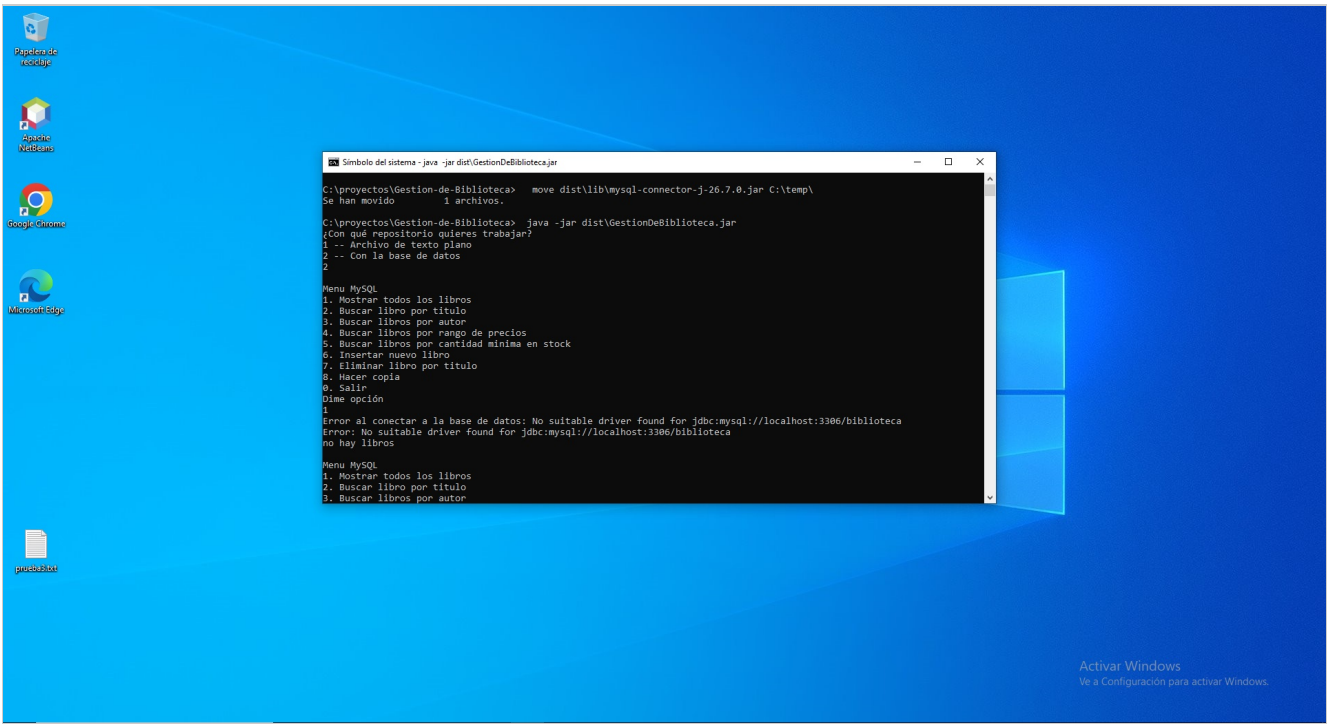
Captura 40. Error «Access denied for user» (contraseña del .env incorrecta).



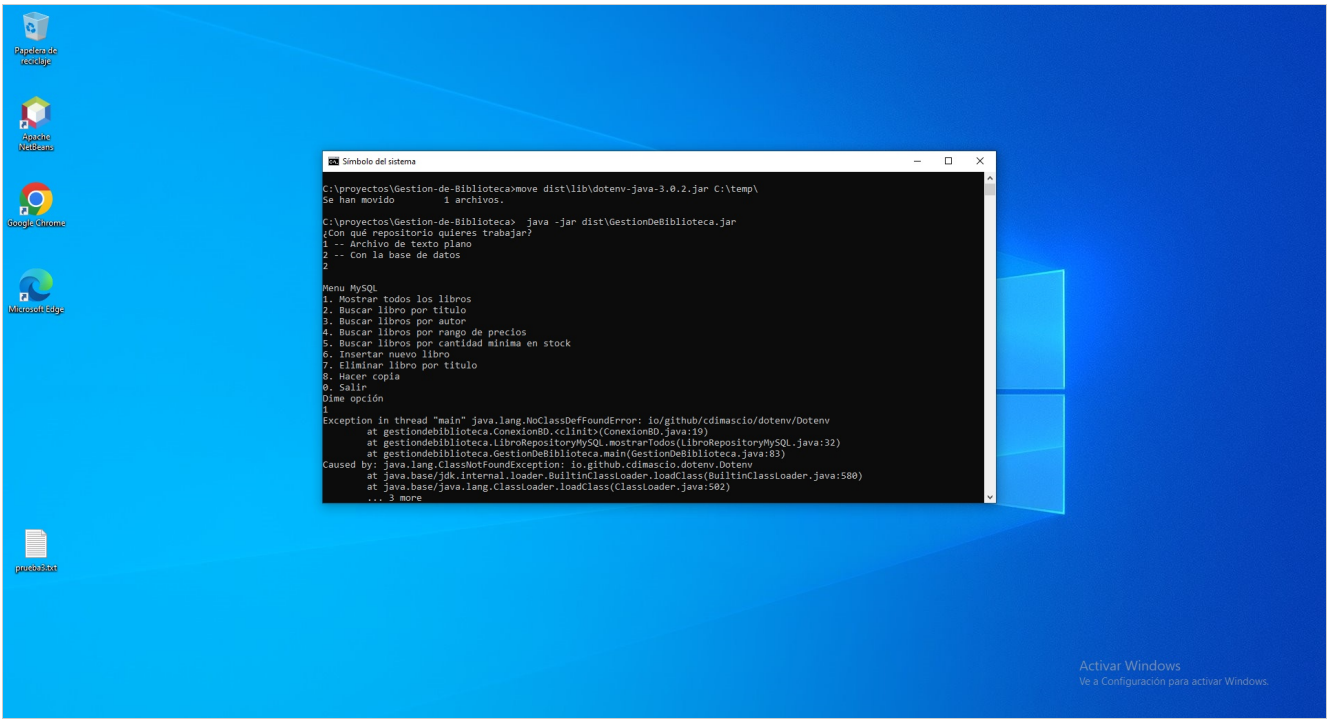
Captura 41. Error «Communications link failure» (servicio MySQL parado).



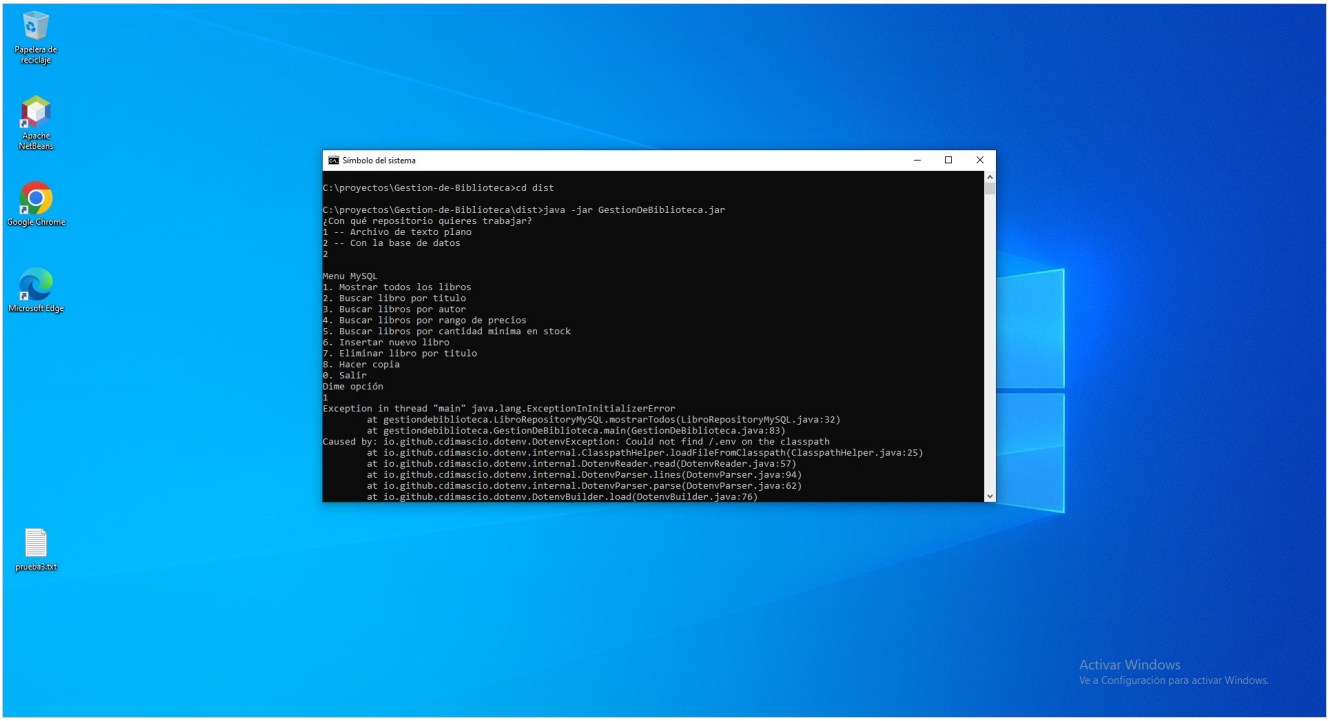
Captura 42. Error «Unknown database» (nombre de base de datos incorrecto en DB_URL).



Captura 43. Error «No suitable driver found» (falta el conector MySQL).

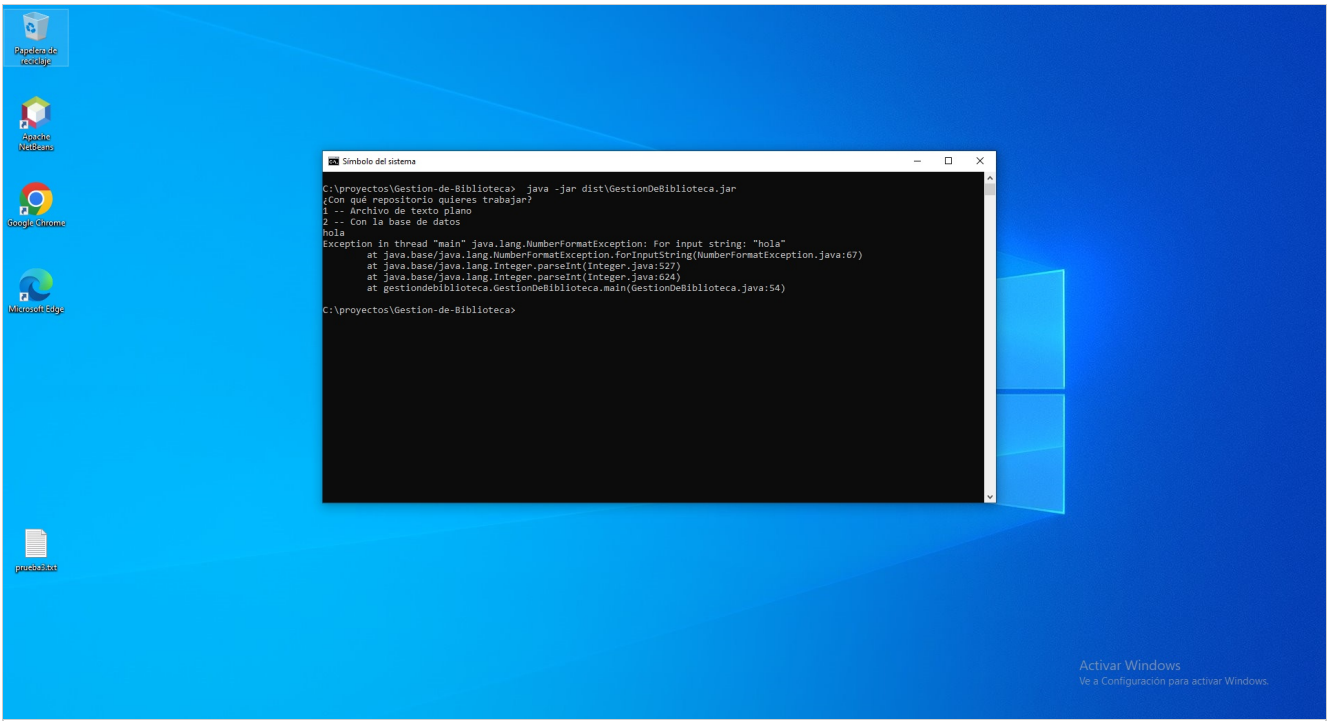


Captura 44. Error «NoClassDefFoundError» de dotenv (falta la librería o el .jar sin lib/).

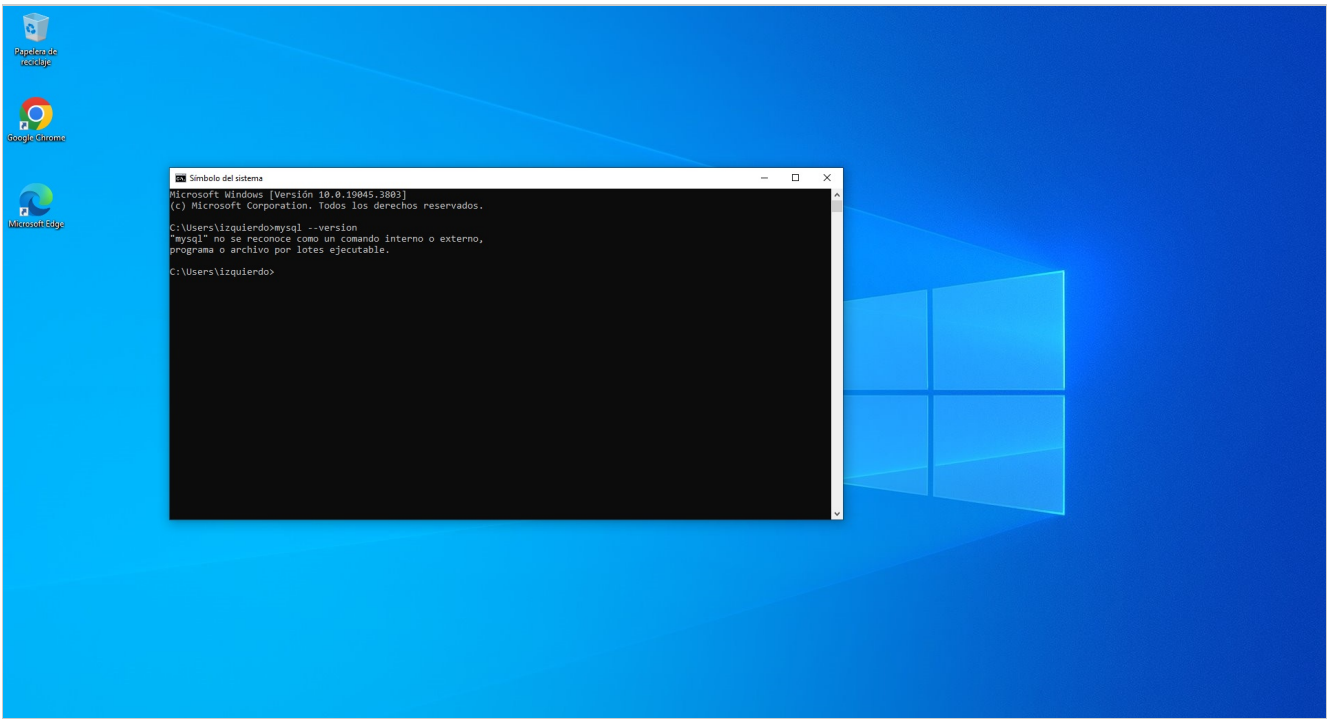


Captura 45. Error «Could not find /.env» (sin archivo .env en el directorio de trabajo).

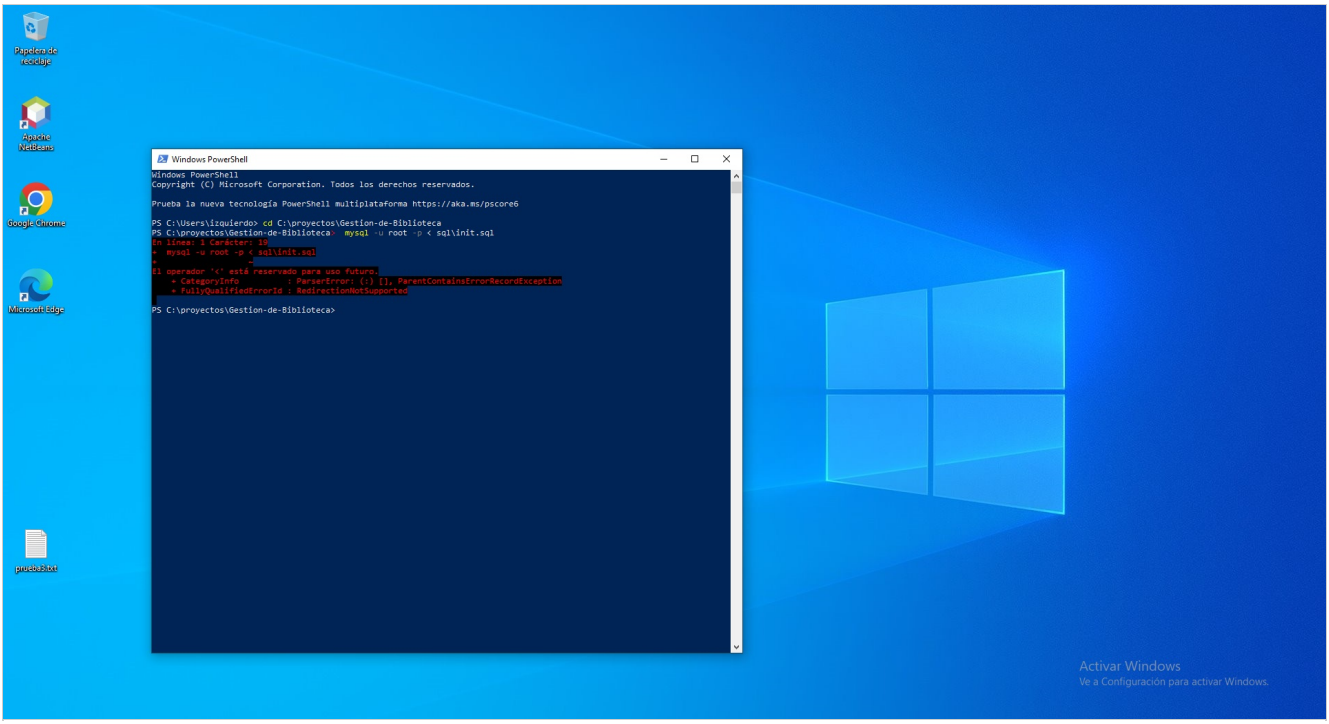
El error `UnsupportedClassVersionError` no lo capturamos porque en la máquina de pruebas solo tenemos el JDK 26. El de `package ... does not exist` está en la captura 15, apartado 6.



Captura 47. Error «NumberFormatException» al escribir texto donde se esperaba un número.



Captura 48. Mensaje «“mysql” no se reconoce» (MySQL fuera del PATH).



Captura 49. Error del operador «<» en PowerShell.